

From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akkradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a small, statically-typed interpreted programming language realised through a four-stage compiler pipeline. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The pipeline consists of a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree, a static type checker that performs inference and rejects ill-typed programs before execution, and a tree-walking interpreter. Both a command-line runner and an interactive desktop interface expose each pipeline stage for inspection. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 73 tests that validates the complete implementation.

CONTENTS

ABSTRACT	ii
1 INTRODUCTION	1
2 LANGUAGE DESIGN AND GRAMMAR	3
2.1 Types	3
2.1.1 Integer	3
2.1.2 Float	3
2.1.3 Boolean	3
2.1.4 String	3
2.1.5 Void	4
2.2 Static Typing	4
2.3 Token Specification	4
2.4 Context-Free Grammar	5
2.4.1 Program Structure	5
2.4.2 Expressions	5
2.4.3 Boolean Expressions	6
2.4.4 Statements	6
2.4.5 Function Definitions and Calls	7
2.5 Operator Precedence and Associativity	7
3 LEXICAL ANALYSIS AND SYMBOL TABLE	8
3.1 Token representation	8
3.2 Lexer implementation	9
3.3 Symbol table and semantic structure	11

3.4	Role of the symbol table in the pipeline	12
4	RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE	14
4.1	Abstract syntax tree representation	14
4.1.1	Expressions	15
4.1.2	Statements	15
4.2	Parser implementation	16
4.2.1	Statement parsing	16
4.2.2	Expression parsing and precedence	17
4.2.3	Helper functions and lookahead	19
4.3	AST printing	19
4.4	Parser and type checker boundary	19
5	TYPE INFERENCE ALGORITHM	20
5.1	Overview	20
5.2	Type Environment	20
5.3	Type Inference Procedure	20
5.4	Inference Rules	22
5.4.1	Literals	22
5.4.2	No Operator Overloading	22
5.4.3	Arithmetic Expressions	23
5.4.4	Boolean Expressions	23
5.4.5	Assignment Statement	24
5.4.6	If-Then-Else	24
5.4.7	While-Loop	24
5.4.8	Function Definition and Call	24
5.5	Type Error Reporting	25
5.6	Example Type Inference Traces	25

6	SYSTEM ARCHITECTURE AND IMPLEMENTATION	26
6.1	Compiler pipeline	26
6.1.1	End-to-end pipeline walkthrough	27
6.1.2	Entry points and shared pipeline	28
6.2	Project structure	28
6.3	Technology stack	29
6.4	Type checker	29
6.4.1	Scope of the implemented type system	29
6.5	Interpreter	30
6.5.1	Scope of the implemented runtime	30
6.6	Graphical user interface	31
6.6.1	Screenshots	32
6.7	Command-line interface	32
7	TESTING AND VERIFICATION	34
7.1	Manual Testing	34
7.1.1	Lexer and Symbol Table	34
7.1.2	Parser and AST	35
7.1.3	Arithmetic Expressions	36
7.1.4	Boolean Expressions	36
7.1.5	Assignment Statements	36
7.1.6	If-Then-Else	36
7.1.7	While-Loop	37
7.1.8	Functions	37
7.1.9	Print	37
7.1.10	Unary Minus	37
7.2	Type Error Detection	37
7.3	Automated Test Suite	38

7.4 Error Handling	39
8 CONCLUSION	40
REFERENCES	41
APPENDIX A: SOURCE CODE	42
APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS	42

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer’s intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a small statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` statement. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser and abstract syntax tree, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the four pipeline stages — tokens, AST, inferred type table, and execution output — for inspection. An automated regression suite of 73 tests verifies correctness across all stages and language features.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents the system architecture and implementation,

including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 0.5. Float values are used exclusively with the dot-suffixed arithmetic operators (`+. .`, `- .`, `* .`, `/ .`), following the Caml Light convention. Integer division (`/`) performs floor division and returns Integer: for example, `10 / 3` evaluates to `3 : Integer`.

The language intentionally does not perform implicit numeric promotion between Integer and Float values. Mixed-type arithmetic such as `1 + 2.5` is rejected by the type checker. Instead, the language follows the Caml Light convention of using separate operator tokens for integer and floating-point arithmetic.

2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so

the value stored internally is the bare character sequence without surrounding quotes. At run-time, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 Void

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker. Therefore, `Void` does not extend the programmer-visible type system required by the assignment.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
1 x = 5;
2 x = "hello";    (* TypeError: Cannot assign String to variable 'x' of
   type Integer *)
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
INT_LITERAL	0, 10, 42
FLOAT_LITERAL	3.14, 20.5
BOOL_LITERAL	true, false
STRING_LITERAL	"hello"
IDENTIFIER	variable and function names such as x, add
Arithmetic operators (integer)	+, -, *, /
Float arithmetic operators	+, -, *, / .
Comparison operators	==, !=
Assignment operator	=
Keywords	if, else, while, def, return, print
Delimiters	() { } , ;

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```

1 program    -> statement* EOF
2 statement  -> assignment
3           | if_stmt
4           | while_stmt
5           | func_def
6           | print_stmt
7           | return_stmt
8           | expr ';'

```

2.4.2 Expressions

```

1 expr       -> additive (('==' | '!=') additive)?
2 additive   -> term (('+' | '-' | '+.' | '-.') term)*
3 term       -> factor (('*' | '/' | '*.' | '/.') factor)*
4 factor     -> '-' factor      (* integer unary negation *)
5           | '-' factor      (* float unary negation *)
6           | INT_LITERAL
7           | FLOAT_LITERAL
8           | STRING_LITERAL
9           | BOOL_LITERAL
10          | IDENTIFIER
11          | IDENTIFIER '(' args? ')',
12          | '(' expr ')',

```

The operators `+`, `-`, `*`, and `/` operate exclusively on Integer operands. The dot-suffixed operators `+.:`, `-.:`, `*.:`, and `/.` operate exclusively on Float operands. This follows the Caml Light convention of using distinct operator tokens for integer and float arithmetic, eliminating the need for implicit numeric promotion. Unary negation is similarly split: `-x` negates an Integer (desugared to `0 - x`), and `-.x` negates a Float (desugared to `0.0 -. x`).

Integer division and division by zero. `/` is floor (integer) division: both operands must be Integer, the result is Integer, and the value is $\lfloor a/b \rfloor$. For example, `7 / 2` evaluates to 3 : Integer. Float division uses the distinct operator `/.` and requires two Float operands. Division by zero (`x / 0` or `x /. 0.0`) is detected at runtime and raises a `RuntimeError`.

2.4.3 Boolean Expressions

Comparison operators `==` and `!=` are the lowest-precedence binary operators and produce Boolean values. Because they are part of the general `expr` rule, Boolean expressions may appear both as control-flow conditions and as ordinary expressions. Their inferred type is Boolean, allowing comparison results to be assigned to variables, passed as arguments, or printed:

```
1 flag = (x == y);      (* flag inferred as Boolean *)
2 print(a != b);        (* prints true/false : Boolean *)
```

Strict arithmetic operands. The specification states that `==` and `!=` operate between two arithmetic expressions. We interpret “arithmetic” strictly as Integer and Float, consistent with the no-overloading rule. Both operands must also share the same numeric type (no implicit coercion). Therefore `Integer == Integer` and `Float == Float` are well-typed; `Integer == Float`, `Boolean == Boolean`, and `String == String` are all rejected at type-check time.

2.4.4 Statements

```
1 assignment -> IDENTIFIER '=' expr ';'
2 if_stmt    -> 'if' '(' expr ')' block ('else' block)?
3 while_stmt -> 'while' '(' expr ')' block
4 print_stmt -> 'print' '(' expr ')' ';'
5 return_stmt -> 'return' expr ';'
6 block      -> '{' statement* '}'
```

Design note: print as a built-in statement

`print` is realised as a built-in statement form rather than a user-callable function. We chose this design for three reasons: (a) the language has no first-class functions or void-returning

user functions, so a built-in callable would be the only member of its kind; (b) the syntactic form `print(expr);` is identical to a function call from the user’s perspective, so there is no usability cost; (c) it allows the `print` operator to emit value : Type deterministically without threading an output type through the function-call machinery. Semantically, `print` behaves as a unary built-in of type $T \rightarrow ()$, but is parsed into a dedicated `Print` AST node and handled as a statement by both the type checker and the interpreter.

2.4.5 Function Definitions and Calls

```

1 func_def -> 'def' IDENTIFIER '(' params? ')' block
2 params   -> IDENTIFIER (',' IDENTIFIER)*
3 args     -> expr (',' expr)*

```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function’s local environment.

Return type uniformity. A function’s return type is inferred from its return statements. All return expressions within a function body must yield the same type; mixed return types are a compile-time error. A function with no `return` statement is assigned the special internal type `Void` and may only be invoked as a standalone statement, not used as an expression. Assigning the result of a `Void` function to a variable (e.g. `x = f();` where `f` returns nothing) is rejected by the type checker.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	<code>==, !=</code>	non-associative (one comparison per expression)
2	<code>+, -, +., -.</code>	left-associative
3 (highest)	<code>*, /, *, ., /. </code>	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative <code>TokenType</code>
Literals	42, 3.14, true, "hello"	INT_LITERAL, FLOAT_LITERAL, BOOL_LITERAL, STRING_LITERAL
Identifiers	x, counter, add	IDENTIFIER
Keywords	if, else, while, def, return, print	keyword-specific token variants
Integer operators	+, -, *, /	PLUS, MINUS, STAR, SLASH
Float operators	+., -., *., /.	PLUS_DOT, MINUS_DOT, STAR_DOT, SLASH_DOT
Other operators	=, ==, !=	ASSIGN, EQUAL_EQUAL, BANG_EQUAL
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	EOF

Token types (from class `TokenType`) are defined using an Enum with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons dur-

ing parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (EOF) token. The EOF token is appended after all input is processed, allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
1 @dataclass(frozen=True)
2 class Token:
3     token_type: TokenType
4     lexeme: str
5     line: int
6     column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an EOF token is added at the end.

Whitespace is skipped, while invalid input results in a `LexerError`.

```
1 def tokenize(self) -> list[Token]:
2     tokens: list[Token] = []
3     while not self._is_at_end():
4         self.start = self.current
5         self.token_column = self.column
6         token = self._scan_token()
7         if token is not None:
```

```

8         tokens.append(token)
9     tokens.append(Token(TokenType.EOF, "", self.line, self.column))
10    return tokens

```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```

1 KEYWORDS: dict[str, TokenType] = {
2     "if": TokenType.IF,
3     "else": TokenType.ELSE,
4     "while": TokenType.WHILE,
5     # ...
6     "true": TokenType.BOOL_LITERAL,
7     "false": TokenType.BOOL_LITERAL,
8 }

```

```

1 # Arithmetic operators with optional trailing '.' for float variant
2 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
3     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
4     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
5     "*": (TokenType.STAR, TokenType.STAR_DOT),
6     "/": (TokenType.SLASH, TokenType.SLASH_DOT),
7 }

```

When one of these characters is encountered, the lexer checks the next character: if it is a `.` (and the character after that is not a digit), the dot-suffixed float variant is emitted; otherwise the plain integer operator is emitted. This lookahead prevents the `.` in a float literal such as `3.14` from being consumed as part of an operator.

```

1 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
2     "=": TokenType.ASSIGN,
3     "(": TokenType.LPAREN,
4     ")": TokenType.RPAREN,
5     # ...
6 }

```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
<code>tokenize()</code>	Iterates through source, scans tokens, appends EOF	ordered token list
<code>_scan_token()</code>	Dispatches recognition based on character and lookahead	next token or skip
<code>_number()</code>	Parses numeric sequences, distinguishes int vs float	numeric literal token
<code>_string()</code>	Parses quoted text, checks termination	string token or error
<code>_identifier()</code>	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
<code>KEYWORDS</code> map	Maps reserved words to token types	token type selection
<code>SINGLE_CHAR_TOKENS</code> map	Maps single-character symbols	token type selection
<code>_advance()</code> , <code>_peek()</code> , <code>_match()</code>	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```

1 def lookup(self, name: str) -> Symbol:
2     symbol = self._symbols.get(name)
3     if symbol is not None:
4         return symbol
5     if self.parent is not None:
6         return self.parent.lookup(name)
7     raise SymbolTableError(f"Undefined symbol: {name}")

```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as `Integer`, `Float`, `Boolean`, `String`, and `Void`. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
<code>LanguageType</code> enum	<code>Integer</code> , <code>Float</code> , <code>Boolean</code> , <code>String</code> , <code>Void</code>	Defines the type system
<code>Symbol</code> (base)	Name and declared or inferred type	Base abstraction for entries
<code>VariableSymbol</code>	Variable metadata (including initialization state)	Tracks correct usage before assignment
<code>FunctionSymbol</code>	Parameters and return type	Validates calls and return consistency
<code>lookup(name)</code>	Recursive scope search	Resolves identifiers or reports undefined
<code>SymbolTableError</code>	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```

1 def run_pipeline(source_code: str) -> PipelineResult:
2     tokens = Lexer(source_code).tokenize()
3     tree = Parser(tokens).parse()
4     ast_text = ASTPrinter().print(tree)
5     checked_scope = TypeChecker().check(tree)
6     outputs: list[str] = []
7     Interpreter(output_callback=outputs.append).execute(tree)
8     return PipelineResult(
9         tokens=tokens,

```

```
10     ast_text=ast_text ,
11     checked_scope=checked_scope ,
12     outputs=outputs ,
13 )
```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical Program AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed as ordinary lowest-precedence expressions, so Boolean results can appear in conditions, assignments, function arguments, and `print(...)` expressions.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (`line`, `column`). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
1 @dataclass
2 class ASTNode:
3     """Base class for every AST node. Carries source location for
4         diagnostics."""
5     line: int = field(default=0, kw_only=True)
6     column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
1 if tok.token_type == TokenType.STRING_LITERAL:
2     self._advance()
3     return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
1 @dataclass
2 class BinaryOp(ASTNode):
3     """An arithmetic or comparison binary operation.
4
5     op is one of: '+', '-', '*', '/', '+.', '-.', '*.', '/.', '==', '!=',
6     Example:  a + b    x == 0    a +. b
7     """
8     op: str
9     left: ASTNode
10    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single Program AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
1 def parse(self) -> Program:
2     """Parse the full token stream and return the root Program node."""
3     line, col = self._peek().line, self._peek().column
4     statements: list[ASTNode] = []
5     while not self._is_at_end():
6         statements.append(self._statement())
7     return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until EOF, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```

1 # assignment: IDENTIFIER "=" expr ";"
2 if (
3     tok.token_type == TokenType.IDENTIFIER
4     and self._peek_next().token_type == TokenType.ASSIGN
5 ):
6     return self._assignment()
7 # fallback: bare expression statement expr ";"
8 node = self._expr()
9 self._expect(TokenType.SEMICOLON, "Expected ';' after expression")

```

4.2.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles comparison operators first (`==`, `!=`) after parsing an additive left-hand side, then `_additive` handles `+`, `-`, `+. .`, and `-. .`; `_term` handles `*`, `/`, `*. .`, and `/. .`; and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code. Float unary negation is handled similarly for `-. .`, producing `BinaryOp(0.0 -. operand)`.

```

1 def _expr(self) -> ASTNode:      # ==, !=
2     ...
3 def _additive(self) -> ASTNode:  # +, -, +. ., -. .
4     ...
5 def _term(self) -> ASTNode:      # *, /, *. ., /. .
6     ...
7 def _factor(self) -> ASTNode:    # unary -, literals, identifiers/calls,
8     ...                          grouped expr

```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_additive` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

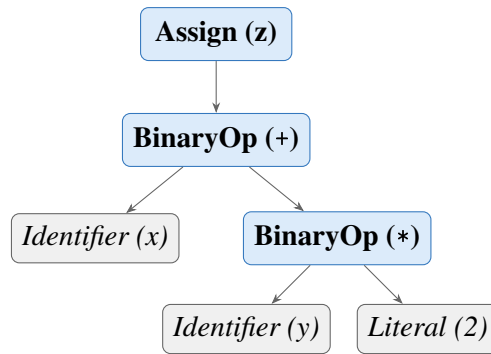


Figure 4.1. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

This structural encoding of precedence eliminates the need for runtime operator-priority resolution and keeps semantic analysis independent of parsing strategy.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Comparison operators (`==`, `!=`) are parsed at the top of the `_expr` method as the lowest-precedence binary operators. Because comparisons are part of the general expression grammar, Boolean results can appear in any expression context — as conditions, assignments, arguments, or print expressions. Semantic enforcement that a condition is truly Boolean is deferred to the type checker.

Figure 4.2 shows the AST for an `if` statement with a comparison condition, illustrating how the parser structures control flow with a Boolean sub-tree.

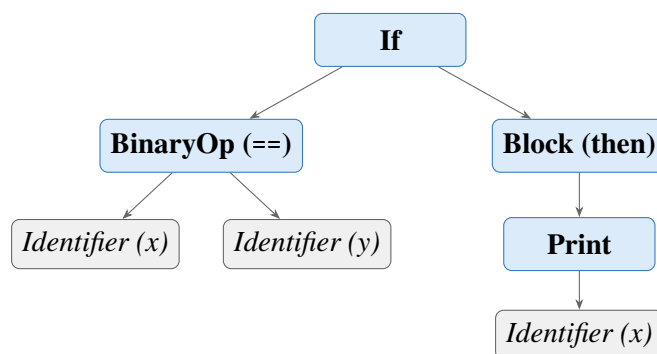


Figure 4.2. AST for `if (x == y) { print(x); }`. The comparison produces a `BinaryOp` node whose inferred type is Boolean, checked by the type checker before execution.

4.2.3 *Helper functions and lookahead*

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token, while comparison operators are recognised in `_expr` after parsing the left additive expression. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure: `Program` and `Block` include statement counts, `BinaryOp` explicitly shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_expr` may syntactically produce a node whose eventual type is not `Boolean`, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \text{Identifier} \rightarrow \text{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm TYPECHECK(*node*, Γ)

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or `TypeError`

1. **If** *node* is a **Literal**: return the corresponding primitive type: `Integer`, `Float`, `Boolean`, or `String`.
2. **If** *node* is an **Identifier**: look up the identifier in Γ . If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
 - Infer the type of the left operand and the right operand.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Integer`; return `Integer`.
 - If the operator is `+`, `-`, `*`, or `/`: require both operands to be `Float`; return `Float`.
 - If the operator is `==` or `!=`: require both operands to be the same arithmetic type; return `Boolean`.
 - Otherwise, report a `TypeError`.
4. **If** *node* is an **Assignment**:
 - Infer the type of the right-hand-side expression.
 - If the variable is not yet in Γ : insert it with the inferred type.
 - Else: verify the new type matches the existing type; if not, report a `TypeError`.
5. **If** *node* is an **If** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the then-block in a child environment.
 - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:
 - Infer the condition type; require it to be `Boolean`.
 - Type-check the loop body in a child environment.
7. **If** *node* is a **FunctionDef**:
 - Register the function name in Γ .
 - Parameter types are initially unknown.
 - Return type is inferred from return statements in the body.

8. If *node* is a **FunctionCall**:

- Infer all argument types.
- If this is the first valid call: store the argument types as the function’s parameter types.
- Else: compare argument types with the stored parameter types.
- Type-check the function body using a local environment.
- Return the inferred function return type.

9. If *node* is a **Print** statement: infer the expression type; accept any of the four supported types.

5.4 Inference Rules

5.4.1 Literals

Literal nodes have direct types:

$$\Gamma \vdash 42 : \text{Integer}$$
$$\Gamma \vdash 3.14 : \text{Float}$$
$$\Gamma \vdash \text{true} : \text{Boolean}$$
$$\Gamma \vdash \text{“hi”} : \text{String}$$

5.4.2 No Operator Overloading

The language does not support operator overloading. Following the Caml Light convention, integer and float arithmetic use entirely separate operator tokens, so every operator has a single, fixed type signature:

- $+$, $-$, $*$, $/$ — require two `Integer` operands; result is `Integer`.
- $+. , -. , *. , /.$ — require two `Float` operands; result is `Float`.
- $+$ is not overloaded for string concatenation; applying it to a `String` is a type error.
- $==$ and $!=$ are not overloaded for `Boolean` or `String` equality.
- No implicit numeric promotion: `Integer + Float` is a type error.

Because every operator has a unique, unambiguous type derivable from its token alone, the compiler can always infer expression types without requiring explicit type declarations from the programmer. This design ensures that operator meaning is determined uniquely from the operator token itself, eliminating semantic ambiguity during type inference and simplifying static analysis.

5.4.3 Arithmetic Expressions

The integer operators `+`, `-`, `*`, and `/` require both operands to be `Integer` and always return `Integer` (including division, which uses integer floor division). The float operators `+`, `-`, `*`, and `/` require both operands to be `Float` and always return `Float`. Mixing an `Integer` and a `Float` with any operator is a type error.

Table 5.1. Arithmetic operator type rules

Left operand	Operator	Right operand	Result type
Integer	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Integer	Integer
Float	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Float	Float
Integer	any	Float	TypeError
Float	any	Integer	TypeError
Any	any	String or Boolean	TypeError

Unary negation

Unary integer negation (`-x`) is desugared by the parser to `0 - x`. Unary float negation (`- . x`) is desugared to `0 . 0 - . x`. The type checker therefore applies the standard binary rules in both cases.

5.4.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two operands of the *same* arithmetic type — either both `Integer` or both `Float`; mixed-type comparisons such as `Integer == Float` are rejected. Comparisons involving `Boolean` or `String` operands are also rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic `Boolean` expressions as equality and inequality between two arithmetic expressions.

5.4.5 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

If $x \notin \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma, x : T \vdash x = e$

If $x : T \in \Gamma$ and $\Gamma \vdash e : T$, then $\Gamma \vdash x = e$

5.4.6 If-Then-Else

The condition of an if statement must be Boolean. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B_{\text{then}} \quad \Gamma'' \vdash B_{\text{else}} \implies \Gamma \vdash \text{if}(c) B_{\text{then}} \text{ else } B_{\text{else}}$$

5.4.7 While-Loop

The condition of a while loop must also be Boolean. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \text{Boolean} \quad \Gamma' \vdash B \implies \Gamma \vdash \text{while}(c) B$$

5.4.8 Function Definition and Call

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the return statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal to the core type-inference algorithm.

5.5 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly.

The format used by the system is:

[line X, col Y] TypeError: message

Examples include assigning a String to an Integer variable, using a non-Boolean condition in an if statement, or supplying the wrong argument type to a function.

5.6 Example Type Inference Traces

Consider the following program fragment:

```
1 x = 3;  
2 y = 4;  
3 z = x + y;  
4 print(z);
```

The checker infers:

- x: Integer (from literal 3)
- y: Integer (from literal 4)
- x + y: Integer (Integer + Integer → Integer)
- z: Integer

For a float example using dot-suffixed operators:

```
1 def addf(a, b) {  
2     return a +. b;  
3 }  
4  
5 result = addf(2.0, 3.5);
```

The first function call infers a: Float and b: Float. The operator +. requires both operands to be Float, so the function return type is inferred as Float. The variable result is also inferred as Float.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

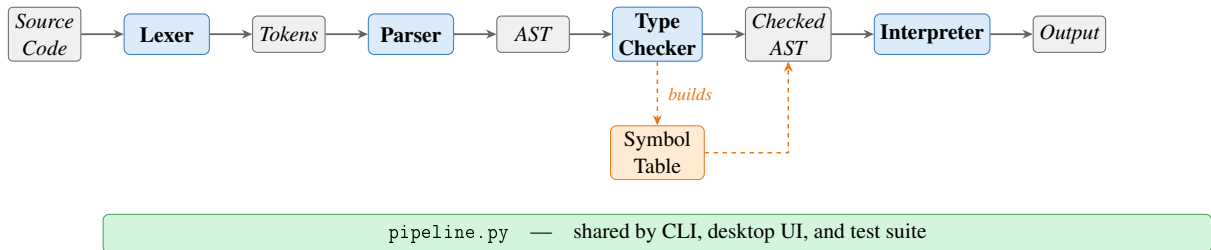


Figure 6.1. Front-end and interpreter pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 End-to-end pipeline walkthrough

The following example traces the complete execution of a three-statement program through all four pipeline stages, illustrating the correspondence between source text, tokens, abstract syntax tree, inferred types, and runtime output.

Source program:

```
1 a = 3;
2 b = 4;
3 print(a + b);
```

Stage 1 — Lexer output (token stream):

```
1 IDENTIFIER      'a'      line 1, col 1
2 ASSIGN          '='      line 1, col 3
3 INT_LITERAL     '3'      line 1, col 5
4 SEMICOLON       ';'      line 1, col 6
5 IDENTIFIER      'b'      line 2, col 1
6 ASSIGN          '='      line 2, col 3
7 INT_LITERAL     '4'      line 2, col 5
8 SEMICOLON       ';'      line 2, col 6
9 PRINT           'print'   line 3, col 1
10 LPAREN         '('      line 3, col 6
11 IDENTIFIER      'a'      line 3, col 7
12 PLUS           '+'      line 3, col 9
13 IDENTIFIER      'b'      line 3, col 11
14 RPAREN         ')'      line 3, col 12
15 SEMICOLON       ';'      line 3, col 13
16 EOF
```

Stage 2 — Parser output (abstract syntax tree):

```
1 Program [3 statement(s)]
2   Assign 'a' =
3     Literal 3 (int)
4   Assign 'b' =
5     Literal 4 (int)
6   Print
7     BinaryOp '+'
8     left:
9       Identifier 'a'
10    right:
11      Identifier 'b'
```

Stage 3 — Type checker output (inferred type environment):

Name	Inferred type
a	Integer
b	Integer

The symbol table records only named symbols (a and b). The type checker additionally infers that the sub-expression `a + b` has type `Integer` (`Integer + Integer → Integer`) during AST traversal, but sub-expression types are not stored in the symbol table.

Stage 4 — Interpreter output:

```
1 7 : Integer
```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured into an AST by the parser, validated and annotated by the type checker, and finally evaluated by the interpreter to produce typed output.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, the CLI and desktop UI can be launched as follows:

```
1 # macOS / Linux
2 python3 main.py          # run CLI (built-in sample or script file)
3 python3 ui.py            # open desktop workbench
4
5 # Windows
6 .\env\Scripts\python.exe main.py
7 .\env\Scripts\python.exe ui.py
```

The CLI prints the four pipeline stages in order: tokens, AST text, type table, and execution output. The desktop workbench uses the same pipeline internally, presenting each stage in a separate tab for inspection.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
<code>main.py</code>	Command-line runner for sample code or a script file
<code>ui.py</code>	Desktop UI for editing and running a script
<code>components/tokens.py</code>	Token classes and token categories
<code>components/lexica.py</code>	Lexical analysis
<code>components/ast_nodes.py</code>	AST node definitions
<code>components/parser.py</code>	Recursive-descent parser
<code>components/ast_printer.py</code>	AST renderer for debugging and reporting
<code>components/symbol_table.py</code>	Scoped symbol table and semantic types
<code>components/type_checker.py</code>	Static type checker and inference
<code>components/interpreter.py</code>	Tree-walking interpreter
<code>components/pipeline.py</code>	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
<code>dataclasses</code>	AST and runtime helper structures
<code>tkinter</code>	Desktop graphical interface
<code>unittest</code>	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,

- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from return statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
1 print(42);           (* 42 : Integer      *)
2 print(3.14);         (* 3.14 : Float       *)
3 print(true);         (* true : Boolean     *)
4 print("plc");        (* "plc" : String    *)
```

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *125970-126690-126687-Ass2-PLC* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready*, a loaded filename, or *Run failed*.

Left panel (source editor). A monospaced `Text` widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A `Notebook` widget presents five tabs, each backed by a scrollable read-only `Text` widget with both horizontal and vertical scrollbars:

- *Execution Output* — the `print()` values produced by stage 4, formatted as `value : Type`. The type label is determined at runtime by inspecting the Python type of the evaluated value; because the type checker guarantees correctness before execution, this always agrees with the statically inferred type;
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *AST* — the indented tree text from stage 2;
- *Type Table* — the symbol table from stage 3 showing each name, kind, inferred type, initialisation state, and parameter list;
- *Errors* — populated only when a pipeline stage fails; shows the error message (e.g. `[line 2, col 5] TypeError: ...`). On a successful run this tab is cleared automatically.

Error handling. If any pipeline stage raises an exception, the error message is written exclusively to the *Errors* tab and the remaining tabs are cleared. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

Keyboard shortcuts. `F5` and `Ctrl+Enter` are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

Figures 6.2 and 6.3 show the desktop workbench and CLI output.



Figure 6.2. Desktop workbench — five output tabs. Errors are reported in the dedicated *Errors* tab; all other tabs are cleared on failure.

6.7 Command-line interface

The CLI runner (`main.py`) accepts an optional script path; without one it runs the built-in sample and prints four labelled stages to `stdout`. Errors are reported cleanly to `stderr`: if the specified file does not exist, the runner prints `ERROR: File not found: <path>` and exits with code 1. If any pipeline stage raises a language error (`LexerError`, `ParseError`, `TypeCheckError`, or `InterpreterError`), the message is printed as `ERROR: [line X, col Y]`

<ErrorType>: <detail> and the process exits with code 1. In both cases no Python traceback is shown to the user.

```
PS C:\Users\johndoe\Desktop\125970-126690-126687-Assignment2-PLC> Get-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned ; & C:\Users\johndoe\Desktop\125970-126690-126687-Assignment2-PLC\venv\Scripts\python.exe python main.py
[...]
```

STAGE 1 - TOKENS

Token	Line	Col
def	1	1
IDENTIFIER	1	1
add	1	5
IDENTIFIER	1	5
LPAREN	1	8
IDENTIFIER	2	1
pi	2	9
IDENTIFIER	2	9
COMMA	2	10
IDENTIFIER	2	11
half	2	12
IDENTIFIER	2	12
RPAREN	2	13
IDENTIFIER	3	1
result	3	5
IDENTIFIER	3	5
ASSIGN	3	10
IDENTIFIER	4	1
pi	4	14
IDENTIFIER	4	14
PLUS	4	16
IDENTIFIER	4	18
half	4	19
SEMICOLON	4	20
PRINT	4	25
LPAREN	4	26
IDENTIFIER	4	27
result	4	27
RPAREN	4	28
SEMICOLON	4	29
RETURN	4	34
IDENTIFIER	4	35
result	4	35
SEMICOLON	4	36
RPAREN	4	37
DEF	4	38
add	4	42
IDENTIFIER	4	42
LPAREN	4	45
IDENTIFIER	4	46
pi	4	47
COMMA	4	48
IDENTIFIER	4	49
half	4	50
RPAREN	4	51
LPAREN	4	52
IDENTIFIER	4	53
result	4	53
ASSIGN	4	54
IDENTIFIER	4	55
pi	4	55

(a) Stage 1: token stream.

```
Assign 'w' =
  Functioncall 'addf' [2 arg(s)]
  arg[0]:
    Identifier 'pi'
  arg[1]:
    Identifier 'half'
Print
  Identifier 'w'
```

STAGE 3 - TYPE CHECK

Name	Kind	Type	Initialized	Parameters
add	function	Integer	-	a: Integer, b: Integer
addf	function	Float	-	a: Float, b: Float
x	variable	Integer	yes	-
y	variable	Integer	yes	-
pi	variable	Float	yes	-
half	variable	Float	yes	-
name	variable	String	yes	-
flag	variable	Boolean	yes	-
i	variable	Integer	yes	-
z	variable	Integer	yes	-
w	variable	Float	yes	-

STAGE 4 - EXECUTION

```
"plc" : String
3 : Integer
2 : Integer
1 : Integer
30 : Integer
30 : Integer
3.64 : Float
3.64 : Float
```

(b) Stage 3 type table and Stage 4 output.

Figure 6.3. CLI run of python main.py on the built-in sample.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table 7.1 summarises the categories covered.

Table 7.1. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate integer and float expressions separately	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true` and `false` are emitted as `BOOL_LITERAL` tokens, the six reserved words (`if`, `else`, `while`, `def`, `return`, `print`) as their respective keyword token types, and non-keyword names as `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```

1 IDENTIFIER  'x'      line=1 col=1
2 ASSIGN      '='      line=1 col=3
3 INT_LITERAL '10'     line=1 col=5
4 SEMICOLON   ';'      line=1 col=7

```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 *Parser and AST*

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_additive/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError`: messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested with integer-only and float-only programs to verify operator precedence and strict type separation. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixing integer and float operands correctly produced a type error, confirming the no-overloading design. The test suite explicitly verifies that mixed-type arithmetic is rejected:

- `1 + 2` — valid Integer arithmetic.
- `2.0 +. 3.5` — valid Float arithmetic.
- `5.0 -. 1.5`, `2.0 *. 3.0`, `9.0 /. 4.0` — valid Float arithmetic for the other dot-suffixed operators.
- `1 + 2.5` — `TypeError: operator + requires two Integer operands`.
- `2 +. 3` — `TypeError: operator +. requires two Float operands`.
- `10 / 0` and `10.0 /. 0.0` — `InterpreterError: division by zero, raised at runtime`.
- `1.0 +. 2.0 *. 3.0` — evaluates to `7.0` confirming `*. binds more tightly than +.`

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of `==` and `!=` must be arithmetic; supplying a Boolean or String operand produced a source-positioned `TypeCheckError` before any execution.

7.1.5 Assignment Statements

Assignments were tested for both first-use type inference and reassignment consistency. Assigning `5` to a variable and subsequently assigning `"hello"` to the same variable correctly produced a type error, confirming that the inferred type is fixed after first assignment.

7.1.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.1.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false, with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.1.8 Functions

Function tests covered value parameter passing, local scope execution, and return value propagation. The type checker correctly inferred parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `3.14 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -. 3.14`), negated float variables (`y = -. x`), negated integer variables (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value. Note that float negation uses the `-.` token, consistent with the Caml Light convention; using plain `-` on a `Float` operand is rejected by the type checker.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
1 # Assignment type mismatch
2 [line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
   Integer
3
4 # Non-Boolean if condition
5 [line 1, col 4] TypeError: If condition must have type Boolean
```

```

6
7 # Wrong argument type for function
8 [line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String

```

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.2. Automated test coverage by class

Test class	Stage covered	Tests
<code>LexerTests</code>	Tokenisation (<code>lexica.py</code>)	12
<code>SymbolTableTests</code>	Scoped symbol table (<code>symbol_table.py</code>)	7
<code>ParserTests</code>	Parsing and precedence (<code>parser.py</code>)	6
<code>TypeCheckerTests</code>	Static type inference (<code>type_checker.py</code>)	22
<code>IntegrationTests</code>	Full pipeline execution	26
Total		73

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators, and the four float-arithmetic operators `+`, `-`, `*`, `/`) is produced correctly, that float literals such as `3.14` are not confused with integer followed by a dot operator, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

ParserTests verify structural correctness of parsing. The 6 tests cover missing-semicolon and unclosed-brace error reporting, integer operator precedence (`*` before `+`), float operator precedence (`*` before `+`), left-associativity of arithmetic operators, and parenthesised expressions overriding precedence.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 22 tests cover integer and float arithmetic (all four dot-suffixed operators `+`, `-`, `*`, `/` each verified to produce `Float`), integer floor division producing `Integer`, strict type separation (mixed `Integer/Float`

rejected for both arithmetic and comparison operators, and integers used with a dot operator rejected), string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types (including `-.` applied to a float variable), division by zero for both `/` and `/.` (each must raise `InterpreterError`), both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, comparison results assigned to variables and printed directly, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
1 # macOS / Linux
2 python3 -m unittest discover -s tests -v
3
4 # Windows
5 .\venv\Scripts\python.exe -m unittest discover -s tests -v
```

All 73 tests pass.

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a complete, statically-typed interpreted language supporting Integer, Float, Boolean, and String types, with arithmetic, comparisons, assignments, if-then-else, while-loops, functions with value parameter passing, and a built-in `print()` function.

The system follows a four-stage pipeline: lexer, parser, type checker, and interpreter. Variable and function types are inferred without explicit declarations; programs that violate type rules are rejected before execution. For uncalled functions, the type checker performs a post-pass to infer parameter types from body usage, ensuring type errors are always caught.

The 73 automated tests across five classes — lexical analysis, symbol table, parsing, type checking, and full pipeline — confirm correct behaviour for all language features. Both a command-line runner and a desktop UI are provided.

The project demonstrates how lexical analysis, parsing, semantic analysis, type inference, and interpretation can be integrated into a coherent compiler pipeline with clear separation of responsibilities between stages. The implementation prioritised semantic clarity, deterministic type inference, and modular compiler structure over advanced language features. Future work could extend the language with recursive functions, richer data structures, and code generation.

REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>

The project can be run from the command line or the desktop UI:

```
1 # macOS / Linux
2 python3 main.py
3 python3 ui.py
4 python3 -m unittest discover -s tests -v
5
6 # Windows
7 .\venv\Scripts\python.exe main.py
8 .\venv\Scripts\python.exe ui.py
9 .\venv\Scripts\python.exe -m unittest discover -s tests -v
```

No external Python dependencies are required.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Contributions
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment, if, while, function, and <code>print()</code> execution; unary minus inference; pipeline integration (<code>pipeline.py</code>); automated test suite (73 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic and boolean expressions; assignment, if-then-else, while-loop, function definition/call, and <code>print()</code> parsing; unary minus parsing

A.1 Entry Points

```
1 from __future__ import annotations
2
```

```

3 import argparse
4 import sys
5 from pathlib import Path
6
7 from components.interpreter import InterpreterError
8 from components.lexica import LexerError
9 from components.parser import ParseError
10 from components.pipeline import format_stage_output, run_pipeline
11 from components.type_checker import TypeCheckError
12
13
14 DEFAULT_SOURCE = """
15 def add(a, b) {
16     result = a + b;
17     print(result);
18     return result;
19 }
20
21 def addf(a, b) {
22     result = a +. b;
23     print(result);
24     return result;
25 }
26
27 x = 10;
28 y = 20;
29 pi = 3.14;
30 half = 0.5;
31 name = "plc";
32 flag = true;
33
34 if (x != 0) {
35     print(name);
36 } else {
37     print(x);
38 }
39
40 i = 3;
41 while (i != 0) {
42     print(i);
43     i = i - 1;
44 }
45
46 z = add(x, y);
47 print(z);
48
49 w = addf(pi, half);
50 print(w);
51 """
52
53
54 def run_source(source_code: str) -> None:
55     try:

```

```

56     result = run_pipeline(source_code)
57     print(format_stage_output(result))
58 except (LexerError, ParseError, TypeCheckError, InterpreterError)
59 as error:
60     print(f"ERROR: {error}", file=sys.stderr)
61     sys.exit(1)
62
63 def _parse_args() -> argparse.Namespace:
64     parser = argparse.ArgumentParser(description="Run the programming
65         language pipeline.")
66     parser.add_argument("script", nargs="?", help="Optional path to a
67         source file to run")
68     return parser.parse_args()
69
70 def main() -> None:
71     args = _parse_args()
72     if args.script:
73         path = Path(args.script)
74         if not path.exists():
75             print(f"ERROR: File not found: {args.script}", file=sys.
76                 stderr)
77             sys.exit(1)
78         source_code = path.read_text(encoding="utf-8")
79     else:
80         source_code = DEFAULT_SOURCE
81     run_source(source_code)
82
83 if __name__ == "__main__":
84     main()

```

Listing 8.1: main.py — command-line entry point

```

1 from __future__ import annotations
2
3 import tkinter as tk
4 from pathlib import Path
5 from tkinter import filedialog, ttk
6
7 from components.pipeline import format_tokens, run_pipeline
8
9
10 DEFAULT_SOURCE = """def add(a, b) {
11     result = a + b;
12     print(result);
13     return result;
14 }
15
16 def addf(a, b) {
17     result = a +. b;
18     print(result);

```

```

19     return result;
20 }
21
22 x = 10;
23 y = 20;
24 pi = 3.14;
25 half = 0.5;
26 name = \"plc\";
27 flag = true;
28
29 if (x != 0) {
30     print(name);
31 } else {
32     print(x);
33 }
34
35 i = 3;
36 while (i != 0) {
37     print(i);
38     i = i - 1;
39 }
40
41 z = add(x, y);
42 print(z);
43
44 w = addf(pi, half);
45 print(w);
46 """
47
48
49 class LanguageWorkbench:
50     def __init__(self) -> None:
51         self.root = tk.Tk()
52         self.root.title("125970-126690-126687-Ass2-PLC")
53         self.root.geometry("1200x760")
54         self.current_file: Path | None = None
55
56         self._build_layout()
57         self.source_text.insert("1.0", DEFAULT_SOURCE)
58         self.root.bind("<F5>", lambda _e: self._run_source())
59         self.root.bind("<Control-Return>", lambda _e: self._run_source
60             ())
61
62     def _build_layout(self) -> None:
63         toolbar = ttk.Frame(self.root, padding=10)
64         toolbar.pack(fill=tk.X)
65
66         ttk.Button(toolbar, text="Open Script", command=self.
67             _open_script).pack(side=tk.LEFT)
68         ttk.Button(toolbar, text="Run", command=self._run_source).pack(
69             side=tk.LEFT, padx=(8, 0))
70         ttk.Button(toolbar, text="Clear Output", command=self.
71             _clear_output).pack(side=tk.LEFT, padx=(8, 0))

```

```

68     self.status_var = tk.StringVar(value="Ready")
69     ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.
70         RIGHT)
71
72     body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
73     body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
74
75     left_panel = ttk.Frame(body, padding=8)
76     right_panel = ttk.Frame(body, padding=8)
77     body.add(left_panel, weight=1)
78     body.add(right_panel, weight=1)
79
80     ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
81     src_frame = ttk.Frame(left_panel)
82     src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
83     self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=(
84         "Consolas", 11))
85     src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=
86         self.source_text.yview)
87     src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL,
88         command=self.source_text.xview)
89     self.source_text.configure(yscrollcommand=src_vsb.set,
90         xscrollcommand=src_hsb.set)
91     src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
92     src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
93     self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
94
95     notebook = ttk.Notebook(right_panel)
96     notebook.pack(fill=tk.BOTH, expand=True)
97
98     self.output_text = self._make_output_tab(notebook, "Execution
99         Output")
100     self.tokens_text = self._make_output_tab(notebook, "Tokens")
101     self.ast_text = self._make_output_tab(notebook, "AST")
102     self.types_text = self._make_output_tab(notebook, "Type Table")
103     self.error_text = self._make_output_tab(notebook, "Errors")
104
105     def _make_output_tab(self, notebook: ttk.Notebook, title: str) ->
106         tk.Text:
107         frame = ttk.Frame(notebook, padding=8)
108         notebook.add(frame, text=title)
109         text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11),
110             state=tk.DISABLED)
111         vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.
112             yview)
113         hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.
114             xview)
115         text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
116         vsb.pack(side=tk.RIGHT, fill=tk.Y)
117         hsb.pack(side=tk.BOTTOM, fill=tk.X)
118         text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
119         return text

```

```

111
112 def _open_script(self) -> None:
113     file_path = filedialog.askopenfilename(
114         title="Open Script File",
115         filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files"
116             , "*.*")],
117     )
118     if not file_path:
119         return
120
121     path = Path(file_path)
122     self.current_file = path
123     self.source_text.delete("1.0", tk.END)
124     self.source_text.insert("1.0", path.read_text(encoding="utf-8")
125     )
126     self.status_var.set(f"Loaded {path.name}")
127
128 def _run_source(self) -> None:
129     source = self.source_text.get("1.0", tk.END)
130     try:
131         result = run_pipeline(source)
132     except Exception as error:
133         self._write_text(self.error_text, str(error))
134         self._write_text(self.output_text, "")
135         self._write_text(self.tokens_text, "")
136         self._write_text(self.ast_text, "")
137         self._write_text(self.types_text, "")
138         self.status_var.set("Run failed")
139         return
140
141     self._write_text(self.output_text, "\n".join(result.outputs) if
142         result.outputs else "(no output)")
143     self._write_text(self.tokens_text, format_tokens(result.tokens)
144     )
145     self._write_text(self.ast_text, result.ast_text)
146     self._write_text(self.types_text, result.checked_scope.
147         format_table())
148     self._write_text(self.error_text, "")
149
150     current_label = self.current_file.name if self.current_file is
151         not None else "editor contents"
152     self.status_var.set(f"Ran {current_label}")
153
154 def _clear_output(self) -> None:
155     self._write_text(self.output_text, "")
156     self._write_text(self.tokens_text, "")
157     self._write_text(self.ast_text, "")
158     self._write_text(self.types_text, "")
159     self._write_text(self.error_text, "")
160     self.status_var.set("Output cleared")
161
162 @staticmethod
163 def _write_text(widget: tk.Text, content: str) -> None:

```

```

158         widget.config(state=tk.NORMAL)
159         widget.delete("1.0", tk.END)
160         widget.insert("1.0", content)
161         widget.config(state=tk.DISABLED)
162
163     def run(self) -> None:
164         self.root.mainloop()
165
166
167 if __name__ == "__main__":
168     LanguageWorkbench().run()

```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from enum import Enum, auto
5
6
7 class TokenType(Enum):
8     # Literals
9     INT_LITERAL = auto()
10    FLOAT_LITERAL = auto()
11    BOOL_LITERAL = auto()
12    STRING_LITERAL = auto()
13
14    # Identifier
15    IDENTIFIER = auto()
16
17    # Keywords
18    IF = auto()
19    ELSE = auto()
20    WHILE = auto()
21    DEF = auto()
22    RETURN = auto()
23    PRINT = auto()
24
25    # Operators
26    PLUS = auto()
27    MINUS = auto()
28    STAR = auto()
29    SLASH = auto()
30    PLUS_DOT = auto()    # +. (float addition)
31    MINUS_DOT = auto()   # -. (float subtraction)
32    STAR_DOT = auto()    # *. (float multiplication)
33    SLASH_DOT = auto()   # /. (float division)
34    EQUAL_EQUAL = auto()
35    BANG_EQUAL = auto()
36    ASSIGN = auto()

```

```

37
38     # Delimiters
39     LPAREN = auto()
40     RPAREN = auto()
41     LBRACE = auto()
42     RBRACE = auto()
43     COMMA = auto()
44     SEMICOLON = auto()
45
46     EOF = auto()
47
48
49 @dataclass(frozen=True)
50 class Token:
51     token_type: TokenType
52     lexeme: str
53     line: int
54     column: int

```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```

1  from __future__ import annotations
2
3  from typing import Iterator
4
5  from components.tokens import Token, TokenType
6
7
8  KEYWORDS: dict[str, TokenType] = {
9      "if": TokenType.IF,
10     "else": TokenType.ELSE,
11     "while": TokenType.WHILE,
12     "def": TokenType.DEF,
13     "return": TokenType.RETURN,
14     "print": TokenType.PRINT,
15     "true": TokenType.BOOL_LITERAL,
16     "false": TokenType.BOOL_LITERAL,
17 }
18
19
20 # Arithmetic operators that may have a trailing '.' for the float
variant
21 _ARITH_DOT: dict[str, tuple[TokenType, TokenType]] = {
22     "+": (TokenType.PLUS, TokenType.PLUS_DOT),
23     "-": (TokenType.MINUS, TokenType.MINUS_DOT),
24     "*": (TokenType.STAR, TokenType.STAR_DOT),
25     "/": (TokenType.SLASH, TokenType.SLASH_DOT),
26 }
27
28 SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
29     "=": TokenType.ASSIGN,
30     "(": TokenType.LPAREN,
31     ")": TokenType.RPAREN,

```

```

32     "{": TokenType.LBRACE,
33     "}": TokenType.RBRACE,
34     ",": TokenType.COMMA,
35     ";": TokenType.SEMICOLON,
36 }
37
38
39 class LexerError(Exception):
40     pass
41
42
43 class Lexer:
44     def __init__(self, source: str) -> None:
45         self.source = source
46         self.start = 0
47         self.current = 0
48         self.line = 1
49         self.column = 1
50         self.token_column = 1
51
52     def tokenize(self) -> list[Token]:
53         tokens: list[Token] = []
54         while not self._is_at_end():
55             self.start = self.current
56             self.token_column = self.column
57             token = self._scan_token()
58             if token is not None:
59                 tokens.append(token)
60         tokens.append(Token(TokenType.EOF, "", self.line, self.column))
61         return tokens
62
63     def _scan_token(self) -> Token | None:
64         char = self._advance()
65
66         # Whitespace handling
67         if char in {" ", "\t", "\r"}:
68             return None
69         if char == "\n":
70             self.line += 1
71             self.column = 1
72             return None
73
74         # Two-char operators
75         if char == "=":
76             if self._match("=="):
77                 return self._make_token(TokenType.EQUAL_EQUAL)
78             return self._make_token(TokenType.ASSIGN)
79         if char == "!":
80             if self._match("!="):
81                 return self._make_token(TokenType.BANG_EQUAL)
82             raise LexerError(self._error_message("Unexpected '!' . Did you mean '!='?"))
83

```

```

84     # Arithmetic operators: + - * / (with optional trailing '.'
      # for float variant)
85     arith = _ARITH_DOT.get(char)
86     if arith is not None:
87         int_tok, float_tok = arith
88         # Unary-minus lookahead: '-' followed by a digit is NOT
      # '-' even if
89         # next char is '.' -- we still need to check char after '.'
      # is not digit
90         # (to avoid consuming the '.' of a float literal like 3.14)
      #
91         # Rule: X. is a float operator only when the char after '.'
      # is NOT a digit.
92         if self._peek() == "." and not self._peek_next().isdigit():
93             self._advance() # consume '.'
94             return self._make_token(float_tok)
95         return self._make_token(int_tok)
96
97     # Single-char operators/delimiters
98     single_char_token = SINGLE_CHAR_TOKENS.get(char)
99     if single_char_token is not None:
100         return self._make_token(single_char_token)
101
102     if char == '"':
103         return self._string()
104
105     if char.isdigit():
106         return self._number()
107
108     if self._is_identifier_start(char):
109         return self._identifier()
110
111     raise LexerError(self._error_message(f"Unexpected character: {
      char!r}"))
112
113     def _string(self) -> Token:
114         while self._peek() != '"' and not self._is_at_end():
115             if self._peek() == "\n":
116                 self.line += 1
117                 self.column = 1
118             self._advance()
119
120         if self._is_at_end():
121             raise LexerError(self._error_message("Unterminated string
      literal"))
122
123         # Closing quote
124         self._advance()
125         return self._make_token(TokenType.STRING_LITERAL)
126
127     def _number(self) -> Token:
128         while self._peek().isdigit():
129             self._advance()

```

```

130
131     is_float = False
132     if self._peek() == "." and self._peek_next().isdigit():
133         is_float = True
134         self._advance()
135         while self._peek().isdigit():
136             self._advance()
137
138     if is_float:
139         return self._make_token(TokenType.FLOAT_LITERAL)
140     return self._make_token(TokenType.INT_LITERAL)
141
142     def _identifier(self) -> Token:
143         while self._is_identifier_part(self._peek()):
144             self._advance()
145
146         lexeme = self._current_lexeme()
147         token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
148         return self._make_token(token_type)
149
150     def _current_lexeme(self) -> str:
151         return self.source[self.start : self.current]
152
153     def _make_token(self, token_type: TokenType) -> Token:
154         return Token(token_type, self._current_lexeme(), self.line,
155                     self.token_column)
156
157     def _advance(self) -> str:
158         char = self.source[self.current]
159         self.current += 1
160         self.column += 1
161         return char
162
163     def _match(self, expected: str) -> bool:
164         if self._is_at_end():
165             return False
166         if self.source[self.current] != expected:
167             return False
168
169         self.current += 1
170         self.column += 1
171         return True
172
173     def _peek(self) -> str:
174         if self._is_at_end():
175             return "\0"
176         return self.source[self.current]
177
178     def _peek_next(self) -> str:
179         if self.current + 1 >= len(self.source):
180             return "\0"
181         return self.source[self.current + 1]

```

```

182     def _is_at_end(self) -> bool:
183         return self.current >= len(self.source)
184
185     @staticmethod
186     def _is_identifier_start(char: str) -> bool:
187         return char.isalpha() or char == "_"
188
189     @staticmethod
190     def _is_identifier_part(char: str) -> bool:
191         return char.isalnum() or char == "_"
192
193     def _error_message(self, message: str) -> str:
194         return f"[line {self.line}, col {self.token_column}] {message}"
195
196
197 def iter_tokens(source: str) -> Iterator[Token]:
198     yield from Lexer(source).tokenize()

```

Listing 8.4: components/lexica.py — lexical analyser

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from typing import Optional
5
6
7 #
8     -----
9
10 # Base
11 #
12     -----
13
14 @dataclass
15 class ASTNode:
16     """Base class for every AST node. Carries source location for
17     diagnostics."""
18     line: int = field(default=0, kw_only=True)
19     column: int = field(default=0, kw_only=True)
20
21 #
22     -----
23
24 # Expressions
25 #
26     -----
27
28
29 @dataclass
30 class Literal(ASTNode):
31     """An integer, float, boolean, or string literal.

```

```

25
26     Examples:  42    3.14    true    "hello"
27     """
28     value: int | float | bool | str
29
30
31 @dataclass
32 class Identifier(ASTNode):
33     """A variable or function name used as an expression.
34
35     Example:  x
36     """
37     name: str
38
39
40 @dataclass
41 class BinaryOp(ASTNode):
42     """An arithmetic or comparison binary operation.
43
44     op is one of: '+' '-' '*' '/' '+.' '-.' '*.' '/.' '==' '!=',
45     Example:  a + b    x == 0    a +. b
46     """
47     op: str
48     left: ASTNode
49     right: ASTNode
50
51
52 @dataclass
53 class FunctionCall(ASTNode):
54     """A call to a user-defined function.
55
56     Example:  add(1, 2)
57     """
58     name: str
59     args: list[ASTNode] = field(default_factory=list)
60
61
62 #
63
64 # -----
65
66 # Statements
67 #
68 # -----
69
70 @dataclass
71 class Assign(ASTNode):
72     """Variable assignment (first use infers type; later uses must
73     match).
74
75     Example:  x = 10;
76     """
77     name: str

```

```

73     value: ASTNode
74
75
76 @dataclass
77 class Print(ASTNode):
78     """Built-in print statement.
79
80     Example:  print(x);
81     """
82     expr: ASTNode
83
84
85 @dataclass
86 class Return(ASTNode):
87     """Return statement inside a function body.
88
89     Example:  return result;
90     """
91     expr: ASTNode
92
93
94 @dataclass
95 class Block(ASTNode):
96     """A brace-enclosed sequence of statements.
97
98     Example:  { x = 1; print(x); }
99     """
100     statements: list[ASTNode] = field(default_factory=list)
101
102
103 @dataclass
104 class If(ASTNode):
105     """If / optional else control flow.
106
107     Example:  if (x != 0) { print(x); } else { print(0); }
108     """
109     condition: ASTNode
110     then_block: Block
111     else_block: Optional[Block]
112
113
114 @dataclass
115 class While(ASTNode):
116     """While loop.
117
118     Example:  while (x != 0) { x = x - 1; }
119     """
120     condition: ASTNode
121     body: Block
122
123
124 @dataclass
125 class FunctionDef(ASTNode):

```

```

126     """Function definition.
127
128     Example:  def add(a, b) { return a + b; }
129
130     Note: parameter types are unknown at parse time -- Member 3's type
131           checker
132           will infer them from usage. We store only the parameter names here.
133     """
134     name: str
135     params: list[str]
136     body: Block
137
138 #
139 # Top-level
140 #
141
142 @dataclass
143 class Program(ASTNode):
144     """Root node: the entire source file as a list of statements."""
145     statements: list[ASTNode] = field(default_factory=list)

```

Listing 8.5: components/ast_nodes.py — AST node definitions

```

1 from __future__ import annotations
2
3 from typing import Optional
4
5 from components.tokens import Token, TokenType
6 from components.ast_nodes import (
7     ASTNode,
8     Program,
9     Block,
10    Assign,
11    If,
12    While,
13    FunctionDef,
14    FunctionCall,
15    Print,
16    Return,
17    BinaryOp,
18    Literal,
19    Identifier,
20 )
21
22
23 #

```

```

24 # Parser error
25 #
26
27 class ParseError(Exception):
28     pass
29
30
31 #
32
33 # Parser
34
35 class Parser:
36     """Recursive-descent parser.
37
38     Usage:
39         tokens = Lexer(source).tokenize()
40         tree   = Parser(tokens).parse()    # returns Program node
41     """
42
43     def __init__(self, tokens: list[Token]) -> None:
44         self._tokens = tokens
45         self._pos = 0
46
47     #
48
49     # Public entry point
50
51     def parse(self) -> Program:
52         """Parse the full token stream and return the root Program node
53         ."""
54         line, col = self._peek().line, self._peek().column
55         statements: list[ASTNode] = []
56         while not self._is_at_end():
57             statements.append(self._statement())
58         return Program(statements=statements, line=line, column=col)
59
60     #
61
62     # Statements
63
64

```

```

62
63 def _statement(self) -> ASTNode:
64     """Dispatch to the correct statement parser based on the
65         current token."""
66     tok = self._peek()
67
68     if tok.token_type == TokenType.DEF:
69         return self._func_def()
70
71     if tok.token_type == TokenType.IF:
72         return self._if_stmt()
73
74     if tok.token_type == TokenType.WHILE:
75         return self._while_stmt()
76
77     if tok.token_type == TokenType.RETURN:
78         return self._return_stmt()
79
80     if tok.token_type == TokenType.PRINT:
81         return self._print_stmt()
82
83     # assignment: IDENTIFIER "=" expr ";"
84     if (
85         tok.token_type == TokenType.IDENTIFIER
86         and self._peek_next().token_type == TokenType.ASSIGN
87     ):
88         return self._assignment()
89
90     # fallback: bare expression statement expr ";"
91     node = self._expr()
92     self._expect(TokenType.SEMICOLON, "Expected ';' after
93         expression")
94     return node
95
96 def _assignment(self) -> Assign:
97     """IDENTIFIER '=' expr ';'"""
98     name_tok = self._advance() #
99     IDENTIFIER # '='
100     self._advance()
101     value = self._expr()
102     self._expect(TokenType.SEMICOLON, "Expected ';' after
103         assignment")
104     return Assign(name=name_tok.lexeme, value=value,
105         line=name_tok.line, column=name_tok.column)
106
107 def _if_stmt(self) -> If:
108     """'if' '(' expr ')' block ( 'else' block )?"""
109     tok = self._advance() # 'if'
110     self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
111     condition = self._expr()
112     self._expect(TokenType.RPAREN, "Expected ')' after if condition
113         ")
114     then_block = self._block()

```

```

110     else_block: Optional[Block] = None
111     if self._check(TokenType.ELSE):
112         self._advance() # 'else'
113         else_block = self._block()
114     return If(condition=condition, then_block=then_block,
115             else_block=else_block,
116             line=tok.line, column=tok.column)
117
118 def _while_stmt(self) -> While:
119     """'while' '(' expr ')' block"""
120     tok = self._advance() # 'while'
121     self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
122     condition = self._expr()
123     self._expect(TokenType.RPAREN, "Expected ')' after while condition")
124     body = self._block()
125     return While(condition=condition, body=body,
126             line=tok.line, column=tok.column)
127
128 def _func_def(self) -> FunctionDef:
129     """'def' IDENTIFIER '(' params? ')' block"""
130     tok = self._advance() # 'def'
131     name_tok = self._expect(TokenType.IDENTIFIER, "Expected function name after 'def'")
132     self._expect(TokenType.LPAREN, "Expected '(' after function name")
133     params = self._params()
134     self._expect(TokenType.RPAREN, "Expected ')' after parameters")
135     body = self._block()
136     return FunctionDef(name=name_tok.lexeme, params=params, body=body,
137             line=tok.line, column=tok.column)
138
139 def _print_stmt(self) -> Print:
140     """'print' '(' expr ')' ';'"""
141     tok = self._advance() # 'print'
142     self._expect(TokenType.LPAREN, "Expected '(' after 'print'")
143     expr = self._expr()
144     self._expect(TokenType.RPAREN, "Expected ')' after print expression")
145     self._expect(TokenType.SEMICOLON, "Expected ';' after print statement")
146     return Print(expr=expr, line=tok.line, column=tok.column)
147
148 def _return_stmt(self) -> Return:
149     """'return' expr ';'"""
150     tok = self._advance() # 'return'
151     expr = self._expr()
152     self._expect(TokenType.SEMICOLON, "Expected ';' after return

```

```

        value")
    return Return(expr=expr, line=tok.line, column=tok.column)

def _block(self) -> Block:
    """'{' statement* '}'"""
    tok = self._expect(TokenType.LBRACE, "Expected '{'")
    statements: list[ASTNode] = []
    while not self._check(TokenType.RBRACE) and not self._is_at_end():
        statements.append(self._statement())
    self._expect(TokenType.RBRACE, "Expected '}' to close block")
    return Block(statements=statements, line=tok.line, column=tok.column)

#
-----

# Parameters (function definition)
#
-----

def _params(self) -> list[str]:
    """IDENTIFIER (',' IDENTIFIER)* -- returns list of param names.
    """
    params: list[str] = []
    if self._check(TokenType.IDENTIFIER):
        params.append(self._advance().lexeme)
        while self._check(TokenType.COMMA):
            self._advance() # ','
            tok = self._expect(TokenType.IDENTIFIER, "Expected
                parameter name after ','")
            params.append(tok.lexeme)
    return params

#
-----

# Expressions (comparison is lowest precedence, then additive,
# then term)
#
-----

def _expr(self) -> ASTNode:
    """additive (('==' | '!=') additive)? -- non-associative
    comparison"""
    node = self._additive()
    if self._check(TokenType.EQUAL_EQUAL) or self._check(TokenType.BANG_EQUAL):
        op_tok = self._advance()
        right = self._additive()
        node = BinaryOp(op=op_tok.lexeme, left=node, right=right,

```

```

189         line=op_tok.line, column=op_tok.column)
190     return node
191
192     def _additive(self) -> ASTNode:
193         """term (('+' | '-' | '+.' | '-.') term)*"""
194         node = self._term()
195         while self._check(TokenType.PLUS) or self._check(TokenType.
196             MINUS) \
197             or self._check(TokenType.PLUS_DOT) or self._check(
198                 TokenType.MINUS_DOT):
199             op_tok = self._advance()
200             right = self._term()
201             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
202                 line=op_tok.line, column=op_tok.column)
203         return node
204
205     def _term(self) -> ASTNode:
206         """factor (('*' | '/' | '*.' | '/.') factor)*"""
207         node = self._factor()
208         while self._check(TokenType.STAR) or self._check(TokenType.
209             SLASH) \
210             or self._check(TokenType.STAR_DOT) or self._check(
211                 TokenType.SLASH_DOT):
212             op_tok = self._advance()
213             right = self._factor()
214             node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
215                 line=op_tok.line, column=op_tok.column)
216         return node
217
218     def _factor(self) -> ASTNode:
219         """
220         '-' factor (unary negation, desugared to 0 - factor)
221         | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
222         | IDENTIFIER ( '(' args? ')' )?
223         | '(' expr ')'
224         """
225         tok = self._peek()
226
227         # Unary minus '-' factor (integer negation, desugared to 0 -
228         # factor)
229         if tok.token_type == TokenType.MINUS:
230             op_tok = self._advance()
231             operand = self._factor()
232             zero = Literal(value=0, line=op_tok.line, column=op_tok.
233                 column)
234             return BinaryOp(op="-", left=zero, right=operand,
235                 line=op_tok.line, column=op_tok.column)
236
237         # Unary minus-dot '-.' factor (float negation, desugared to
238         # 0.0 -. factor)
239         if tok.token_type == TokenType.MINUS_DOT:
240             op_tok = self._advance()
241             operand = self._factor()

```

```

235         zero_f = Literal(value=0.0, line=op_tok.line, column=op_tok
236                             .column)
237         return BinaryOp(op="-. ", left=zero_f, right=operand,
238                             line=op_tok.line, column=op_tok.column)
239
240     # Integer literal
241     if tok.token_type == TokenType.INT_LITERAL:
242         self._advance()
243         return Literal(value=int(tok.lexeme), line=tok.line, column
244                             =tok.column)
245
246     # Float literal
247     if tok.token_type == TokenType.FLOAT_LITERAL:
248         self._advance()
249         return Literal(value=float(tok.lexeme), line=tok.line,
250                             column=tok.column)
251
252     # Boolean literal
253     if tok.token_type == TokenType.BOOL_LITERAL:
254         self._advance()
255         return Literal(value=(tok.lexeme == "true"), line=tok.line,
256                             column=tok.column)
257
258     # String literal (keep the surrounding quotes stripped)
259     if tok.token_type == TokenType.STRING_LITERAL:
260         self._advance()
261         return Literal(value=tok.lexeme[1:-1], line=tok.line,
262                             column=tok.column)
263
264     # Identifier -- could be a plain variable or a function call
265     if tok.token_type == TokenType.IDENTIFIER:
266         self._advance()
267         if self._check(TokenType.LPAREN):
268             # function call
269             self._advance()
270             args = self._args()
271             self._expect(TokenType.RPAREN, "Expected ')' after
272                                     function arguments")
273             return FunctionCall(name=tok.lexeme, args=args,
274                                     line=tok.line, column=tok.column)
275         return Identifier(name=tok.lexeme, line=tok.line, column=
276                             tok.column)
277
278     # Grouped expression '(' expr ')'
279     if tok.token_type == TokenType.LPAREN:
280         self._advance()
281         node = self._expr()
282         self._expect(TokenType.RPAREN, "Expected ')' to close
283                                     grouped expression")
284         return node
285
286     raise ParseError(self._error_at(tok, f"Unexpected token: {tok.
287                                     lexeme!r}"))

```

```

279
280 #
281
282 # Arguments (function call)
283
284 def _args(self) -> list[ASTNode]:
285     """expr (',' expr)*"""
286     args: list[ASTNode] = []
287     if not self._check(TokenType.RPAREN):
288         args.append(self._expr())
289         while self._check(TokenType.COMMA):
290             self._advance() # ','
291             args.append(self._expr())
292     return args
293
294 #
295
296 # Token navigation helpers
297
298 def _peek(self) -> Token:
299     return self._tokens[self._pos]
300
301 def _peek_next(self) -> Token:
302     if self._pos + 1 < len(self._tokens):
303         return self._tokens[self._pos + 1]
304     return self._tokens[-1] # EOF
305
306 def _advance(self) -> Token:
307     tok = self._tokens[self._pos]
308     if not self._is_at_end():
309         self._pos += 1
310     return tok
311
312 def _check(self, token_type: TokenType) -> bool:
313     return self._peek().token_type == token_type
314
315 def _is_at_end(self) -> bool:
316     return self._peek().token_type == TokenType.EOF
317
318 def _expect(self, token_type: TokenType, message: str) -> Token:
319     if self._check(token_type):
320         return self._advance()
321     raise ParseError(self._error_at(self._peek(), message))
322
323 def _error_at(self, tok: Token, message: str) -> str:

```

324

```

    return f"[line {tok.line}, col {tok.column}] ParseError: {
        message}"

```

Listing 8.6: components/parser.py — recursive-descent parser

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass, field
4  from enum import Enum
5
6
7  class LanguageType(Enum):
8      INTEGER = "Integer"
9      FLOAT = "Float"
10     BOOLEAN = "Boolean"
11     STRING = "String"
12     VOID = "Void"
13
14
15     @dataclass
16     class Symbol:
17         name: str
18         symbol_type: LanguageType
19
20
21     @dataclass
22     class VariableSymbol(Symbol):
23         initialized: bool = False
24
25
26     @dataclass
27     class FunctionSymbol(Symbol):
28         parameters: list[tuple[str, LanguageType]] = field(default_factory=
29             list)
30
31     class SymbolTableError(Exception):
32         pass
33
34
35     class SymbolTable:
36         def __init__(self, parent: SymbolTable | None = None) -> None:
37             self.parent = parent
38             self._symbols: dict[str, Symbol] = {}
39
40         def define_variable(self, name: str, symbol_type: LanguageType,
41             initialized: bool = False) -> VariableSymbol:
42             if name in self._symbols:
43                 raise SymbolTableError(f"Symbol already defined in this
44                     scope: {name}")
45             symbol = VariableSymbol(name=name, symbol_type=symbol_type,
46                 initialized=initialized)
47             self._symbols[name] = symbol

```

```

45     return symbol
46
47     def define_function(
48         self,
49         name: str,
50         return_type: LanguageType,
51         parameters: list[tuple[str, LanguageType]],
52     ) -> FunctionSymbol:
53         if name in self._symbols:
54             raise SymbolTableError(f"Symbol already defined in this
55                                     scope: {name}")
56         symbol = FunctionSymbol(name=name, symbol_type=return_type,
57                                 parameters=parameters)
58         self._symbols[name] = symbol
59         return symbol
60
61     def lookup(self, name: str) -> Symbol:
62         symbol = self._symbols.get(name)
63         if symbol is not None:
64             return symbol
65         if self.parent is not None:
66             return self.parent.lookup(name)
67         raise SymbolTableError(f"Undefined symbol: {name}")
68
69     def exists_in_current_scope(self, name: str) -> bool:
70         return name in self._symbols
71
72     def child_scope(self) -> SymbolTable:
73         return SymbolTable(parent=self)
74
75     def set_initialized(self, name: str) -> None:
76         symbol = self.lookup(name)
77         if not isinstance(symbol, VariableSymbol):
78             raise SymbolTableError(f"Cannot initialize non-variable
79                                     symbol: {name}")
80         symbol.initialized = True
81
82     def snapshot(self) -> dict[str, Symbol]:
83         return dict(self._symbols)
84
85     def format_table(self) -> str:
86         lines = [
87             f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized'
88                 '<12} Parameters",
89             "-" * 72,
90         ]
91
92         for name, symbol in self._symbols.items():
93             if isinstance(symbol, FunctionSymbol):
94                 params = ", ".join(f"{param_name}: {param_type.value}"
95                                     for param_name, param_type in symbol.parameters)
96                 lines.append(
97                     f"{name:<12} {'function':<10} {symbol.symbol_type.

```

```

93         value:<10> {'-':<12> {params or '-'}}"
94     )
95     elif isinstance(symbol, VariableSymbol):
96         initialized = "yes" if symbol.initialized else "no"
97         lines.append(
98             f"{name:<12> {'variable':<10> {symbol.symbol_type.
99                 value:<10> {initialized:<12> -"
100         )
101     else:
102         lines.append(f"{name:<12> {'symbol':<10> {symbol.
103             symbol_type.value:<10> {'-':<12> -")
104
105     return "\n".join(lines)

```

Listing 8.7: components/symbol_table.py — scoped symbol table

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  from components.ast_nodes import (
6      ASTNode,
7      Assign,
8      BinaryOp,
9      Block,
10     FunctionCall,
11     FunctionDef,
12     Identifier,
13     If,
14     Literal,
15     Print,
16     Program,
17     Return,
18     While,
19 )
20 from components.symbol_table import FunctionSymbol, LanguageType,
21     SymbolTable, SymbolTableError, VariableSymbol
22
23 class TypeCheckError(Exception):
24     pass
25
26
27 @dataclass
28 class _FunctionState:
29     node: FunctionDef
30     body_checked: bool = False
31
32
33 @dataclass
34 class _FunctionContext:
35     name: str
36     return_type: LanguageType | None = None

```

```

37
38
39 class TypeChecker:
40     def __init__(self) -> None:
41         self.global_scope = SymbolTable()
42         self._functions: dict[str, _FunctionState] = {}
43         self._active_calls: list[str] = []
44
45     def check(self, program: Program) -> SymbolTable:
46         for statement in program.statements:
47             if isinstance(statement, FunctionDef):
48                 self._register_function(statement)
49
50         for statement in program.statements:
51             if isinstance(statement, FunctionDef):
52                 continue
53             self._check_statement(statement, self.global_scope, None)
54
55         self._check_uncalled_functions()
56         return self.global_scope
57
58     def _register_function(self, node: FunctionDef) -> None:
59         if node.name in self._functions:
60             raise TypeCheckError(self._error_at(node, f"Function '{node
61                                     .name}' is already defined"))
62
63         placeholder_parameters = [(param_name, LanguageType.VOID) for
64                                   param_name in node.params]
65         try:
66             self.global_scope.define_function(node.name, LanguageType.
67                                               VOID, placeholder_parameters)
68         except SymbolTableError as error:
69             raise TypeCheckError(self._error_at(node, str(error))) from
70             error
71         self._functions[node.name] = _FunctionState(node=node)
72
73     def _check_statement(
74         self,
75         node: ASTNode,
76         scope: SymbolTable,
77         function_context: _FunctionContext | None,
78     ) -> None:
79         if isinstance(node, Assign):
80             value_type = self._infer_expr_type(node.value, scope)
81             self._assign_variable_type(scope, node, value_type)
82             return
83
84         if isinstance(node, Print):
85             self._infer_expr_type(node.expr, scope)
86             return
87
88         if isinstance(node, Return):
89             if function_context is None:

```

```

86         raise TypeCheckError(self._error_at(node, "'return' is
           only valid inside a function"))
87     expr_type = self._infer_expr_type(node.expr, scope)
88     if function_context.return_type is None:
89         function_context.return_type = expr_type
90     elif function_context.return_type != expr_type:
91         raise TypeCheckError(
92             self._error_at(
93                 node,
94                 f"Function '{function_context.name}' returns
                   both {function_context.return_type.value}
                   and {expr_type.value}",
95             )
96         )
97     return
98
99     if isinstance(node, If):
100         condition_type = self._infer_expr_type(node.condition,
101                                                  scope)
102         if condition_type != LanguageType.BOOLEAN:
103             raise TypeCheckError(self._error_at(node.condition, "If
                   condition must have type Boolean"))
104         self._check_block(node.then_block, scope.child_scope(),
105                           function_context)
106         if node.else_block is not None:
107             self._check_block(node.else_block, scope.child_scope(),
108                               function_context)
109         return
110
111     if isinstance(node, While):
112         condition_type = self._infer_expr_type(node.condition,
113                                                  scope)
114         if condition_type != LanguageType.BOOLEAN:
115             raise TypeCheckError(self._error_at(node.condition, "
                   While condition must have type Boolean"))
116         self._check_block(node.body, scope.child_scope(),
117                           function_context)
118         return
119
120     if isinstance(node, Block):
121         self._check_block(node, scope.child_scope(),
122                           function_context)
123         return
124
125     if isinstance(node, FunctionCall):
126         self._infer_function_call_type(node, scope)
127         return
128
129     raise TypeCheckError(self._error_at(node, f"Unsupported
           statement node: {type(node).__name__}"))
130
131 def _check_block(
132     self,

```

```

127     block: Block,
128     scope: SymbolTable,
129     function_context: _FunctionContext | None,
130 ) -> None:
131     for statement in block.statements:
132         self._check_statement(statement, scope, function_context)
133
134 def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) ->
LanguageType:
135     if isinstance(node, Literal):
136         return self._literal_type(node)
137
138     if isinstance(node, Identifier):
139         try:
140             symbol = scope.lookup(node.name)
141         except SymbolTableError as error:
142             raise TypeCheckError(self._error_at(node, str(error)))
143             from error
144         if not isinstance(symbol, VariableSymbol):
145             raise TypeCheckError(self._error_at(node, f"'{node.name
146             }' is not a variable"))
147         return symbol.symbol_type
148
149     if isinstance(node, FunctionCall):
150         return self._infer_function_call_type(node, scope)
151
152     if isinstance(node, BinaryOp):
153         left_type = self._infer_expr_type(node.left, scope)
154         right_type = self._infer_expr_type(node.right, scope)
155         return self._infer_binary_type(node, left_type, right_type)
156
157     raise TypeCheckError(self._error_at(node, f"Unsupported
158     expression node: {type(node).__name__}"))
159
160 def _infer_function_call_type(self, node: FunctionCall, scope:
SymbolTable) -> LanguageType:
161     function_state = self._functions.get(node.name)
162     if function_state is None:
163         raise TypeCheckError(self._error_at(node, f"Undefined
164         function: {node.name}"))
165     if node.name in self._active_calls:
166         raise TypeCheckError(self._error_at(node, f"Recursive
167         function calls are not supported: {node.name}"))
168
169     argument_types = [self._infer_expr_type(argument, scope) for
170         argument in node.args]
171     function_symbol = self._lookup_function_symbol(node)
172
173     if len(argument_types) != len(function_symbol.parameters):
174         raise TypeCheckError(
175             self._error_at(
176                 node,
177                 f"Function '{node.name}' expects {len(

```

```

172         function_symbol.parameters))} argument(s), got {
173         len(argument_types)}",
174     )
175     declared_parameter_types = [param_type for _, param_type in
176     function_symbol.parameters]
177     if all(param_type == LanguageType.VOID for param_type in
178     declared_parameter_types):
179         function_symbol.parameters = list(zip(function_state.node.
180         params, argument_types, strict=False))
181     else:
182         for index, ((_, expected_type), actual_type) in enumerate(
183         zip(function_symbol.parameters, argument_types, strict=
184         False)):
185             if expected_type != actual_type:
186                 raise TypeCheckError(
187                     self._error_at(
188                         node.args[index],
189                         f"Argument {index + 1} of '{node.name}'
190                         must be {expected_type.value}, got {
191                         actual_type.value}",
192                     )
193                 )
194     )
195
196     if not function_state.body_checked:
197         self._check_function_body(function_state, function_symbol)
198
199     return function_symbol.symbol_type
200
201 def _check_function_body(self, function_state: _FunctionState,
202 function_symbol: FunctionSymbol) -> None:
203     function_context = _FunctionContext(name=function_state.node.
204     name)
205     function_scope = self.global_scope.child_scope()
206     for parameter_name, parameter_type in function_symbol.
207     parameters:
208         function_scope.define_variable(parameter_name,
209         parameter_type, initialized=True)
210
211     self._active_calls.append(function_state.node.name)
212     try:
213         self._check_block(function_state.node.body, function_scope,
214         function_context)
215     finally:
216         self._active_calls.pop()
217
218     function_symbol.symbol_type = function_context.return_type or
219     LanguageType.VOID
220     function_state.body_checked = True
221
222 def _check_uncalled_functions(self) -> None:
223     """After all call-site checks, type-check any function whose

```

```

210         body was never visited."""
211     for name, state in self._functions.items():
212         if not state.body_checked:
213             try:
214                 function_symbol = self.global_scope.lookup(name)
215             except SymbolTableError:
216                 continue
217             if not isinstance(function_symbol, FunctionSymbol):
218                 continue
219             inferred_params = self._infer_param_types_from_body(
220                 state.node)
221             function_symbol.parameters = inferred_params
222             self._check_function_body(state, function_symbol)
223
224     def _infer_param_types_from_body(self, node: FunctionDef) -> list[
225         tuple[str, LanguageType]]:
226         """Infer parameter types by scanning the body for expression
227         contexts.
228
229         Any parameter used as an operand of a float operator (+, -, *,
230         /) is
231         inferred as Float; a parameter used with an integer operator (+
232         - * /
233         == !=) is inferred as Integer. Parameters with no type hint
234         default to
235         Integer.
236         """
237         param_names = set(node.params)
238         inferred: dict[str, LanguageType] = {}
239
240     def scan_expr(n: ASTNode) -> None:
241         if isinstance(n, BinaryOp):
242             if n.op in {"+", "-", "*", "/", "==", "!="}:
243                 for operand in (n.left, n.right):
244                     if isinstance(operand, Identifier) and operand.
245                         name in param_names:
246                         inferred.setdefault(operand.name,
247                             LanguageType.INTEGER)
248             elif n.op in {"+.", "-.", "*.", "/."}:
249                 for operand in (n.left, n.right):
250                     if isinstance(operand, Identifier) and operand.
251                         name in param_names:
252                         inferred.setdefault(operand.name,
253                             LanguageType.FLOAT)
254                 scan_expr(n.left)
255                 scan_expr(n.right)
256             elif isinstance(n, FunctionCall):
257                 for arg in n.args:
258                     scan_expr(arg)
259
260     def scan_stmt(n: ASTNode) -> None:
261         if isinstance(n, Assign):
262             scan_expr(n.value)

```

```

252         elif isinstance(n, Print):
253             scan_expr(n.expr)
254         elif isinstance(n, Return):
255             scan_expr(n.expr)
256         elif isinstance(n, If):
257             scan_expr(n.condition)
258             scan_block(n.then_block)
259             if n.else_block:
260                 scan_block(n.else_block)
261         elif isinstance(n, While):
262             scan_expr(n.condition)
263             scan_block(n.body)
264         elif isinstance(n, Block):
265             scan_block(n)
266         elif isinstance(n, FunctionCall):
267             for arg in n.args:
268                 scan_expr(arg)
269
270     def scan_block(block: Block) -> None:
271         for stmt in block.statements:
272             scan_stmt(stmt)
273
274     scan_block(node.body)
275     return [(param, inferred.get(param, LanguageType.INTEGER)) for
276             param in node.params]
277
278     def _assign_variable_type(self, scope: SymbolTable, node: Assign,
279                               value_type: LanguageType) -> None:
280         if value_type == LanguageType.VOID:
281             raise TypeCheckError(self._error_at(node, f"Cannot assign
282             the result of a void function to '{node.name}'"))
283         try:
284             symbol = scope.lookup(node.name)
285         except SymbolTableError:
286             scope.define_variable(node.name, value_type, initialized=
287             True)
288             return
289
290         if not isinstance(symbol, VariableSymbol):
291             raise TypeCheckError(self._error_at(node, f"'{node.name}'
292             is not a variable"))
293         if symbol.symbol_type != value_type:
294             raise TypeCheckError(
295                 self._error_at(
296                     node,
297                     f"Cannot assign {value_type.value} to variable '{
298                     node.name}' of type {symbol.symbol_type.value}",
299                 )
300             )
301         symbol.initialized = True
302
303     def _infer_binary_type(
304         self,

```

```

299     node: BinaryOp,
300     left_type: LanguageType,
301     right_type: LanguageType,
302 ) -> LanguageType:
303     # Integer operators: both operands must be Integer
304     if node.op in {"+", "-", "*"}:
305         if left_type != LanguageType.INTEGER or right_type !=
306             LanguageType.INTEGER:
307             raise TypeCheckError(
308                 self._error_at(node, f"Operator '{node.op}'
309                     requires two Integer operands")
310             )
311         return LanguageType.INTEGER
312
313     # Integer division: both operands must be Integer, result is
314     Integer
315     if node.op == "/":
316         if left_type != LanguageType.INTEGER or right_type !=
317             LanguageType.INTEGER:
318             raise TypeCheckError(
319                 self._error_at(node, f"Operator '{node.op}'
320                     requires two Integer operands")
321             )
322         return LanguageType.INTEGER
323
324     # Float operators: both operands must be Float
325     if node.op in {"+.", "-.", "*.", "/."}:
326         if left_type != LanguageType.FLOAT or right_type !=
327             LanguageType.FLOAT:
328             raise TypeCheckError(
329                 self._error_at(node, f"Operator '{node.op}'
330                     requires two Float operands")
331             )
332         return LanguageType.FLOAT
333
334     if node.op in {"==", "!="}:
335         if not self._is_numeric(left_type) or not self._is_numeric(
336             right_type):
337             raise TypeCheckError(
338                 self._error_at(
339                     node,
340                     f"Operator '{node.op}' requires two arithmetic
341                         operands",
342                 )
343             )
344         if left_type != right_type:
345             raise TypeCheckError(
346                 self._error_at(
347                     node,
348                     f"Operator '{node.op}' requires operands of the
349                         same type, got {left_type.value} and {
350                             right_type.value}",
351                 )
352             )

```

```

341         )
342         return LanguageType.BOOLEAN
343
344         raise TypeCheckError(self._error_at(node, f"Unsupported
operator: {node.op}"))
345
346     def _lookup_function_symbol(self, node: FunctionCall) ->
FunctionSymbol:
347         try:
348             symbol = self.global_scope.lookup(node.name)
349         except SymbolTableError as error:
350             raise TypeCheckError(self._error_at(node, str(error))) from
error
351         if not isinstance(symbol, FunctionSymbol):
352             raise TypeCheckError(self._error_at(node, f"'{node.name}',
is not a function"))
353         return symbol
354
355     @staticmethod
356     def _is_numeric(language_type: LanguageType) -> bool:
357         return language_type in {LanguageType.INTEGER, LanguageType.
FLOAT}
358
359     @staticmethod
360     def _literal_type(node: Literal) -> LanguageType:
361         if isinstance(node.value, bool):
362             return LanguageType.BOOLEAN
363         if isinstance(node.value, int):
364             return LanguageType.INTEGER
365         if isinstance(node.value, float):
366             return LanguageType.FLOAT
367         if isinstance(node.value, str):
368             return LanguageType.STRING
369         raise TypeCheckError(f"Unsupported literal value: {node.value!r
}")
370
371     @staticmethod
372     def _error_at(node: ASTNode, message: str) -> str:
373         return f"[line {node.line}, col {node.column}] TypeError: {
message}"

```

Listing 8.8: components/type_checker.py — static type checker with inference

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from typing import Callable
5
6 from components.ast_nodes import (
7     ASTNode,
8     Assign,
9     BinaryOp,
10    Block,

```

```

11     FunctionCall,
12     FunctionDef,
13     Identifier,
14     If,
15     Literal,
16     Print,
17     Program,
18     Return,
19     While,
20 )
21 from components.symbol_table import LanguageType
22
23
24 class InterpreterError(Exception):
25     pass
26
27
28 class _ReturnSignal(Exception):
29     def __init__(self, value: object) -> None:
30         self.value = value
31
32
33 @dataclass
34 class _FunctionRuntime:
35     node: FunctionDef
36
37
38 class _Environment:
39     def __init__(self, parent: _Environment | None = None) -> None:
40         self.parent = parent
41         self.values: dict[str, object] = {}
42
43     def define(self, name: str, value: object) -> None:
44         self.values[name] = value
45
46     def get(self, name: str) -> object:
47         if name in self.values:
48             return self.values[name]
49         if self.parent is not None:
50             return self.parent.get(name)
51         raise InterpreterError(f"Undefined variable: {name}")
52
53     def assign(self, name: str, value: object) -> None:
54         if name in self.values:
55             self.values[name] = value
56             return
57         if self.parent is not None and self.parent.contains(name):
58             self.parent.assign(name, value)
59             return
60         self.values[name] = value
61
62     def contains(self, name: str) -> bool:
63         if name in self.values:

```

```

64         return True
65     if self.parent is not None:
66         return self.parent.contains(name)
67     return False
68
69
70 class Interpreter:
71     def __init__(self, output_callback: Callable[[str], None] | None =
None) -> None:
72         self._functions: dict[str, _FunctionRuntime] = {}
73         self._output_callback = output_callback or print
74
75     def execute(self, program: Program) -> None:
76         environment = _Environment()
77         for statement in program.statements:
78             if isinstance(statement, FunctionDef):
79                 self._functions[statement.name] = _FunctionRuntime(node
=statement)
80
81         for statement in program.statements:
82             if isinstance(statement, FunctionDef):
83                 continue
84             self._execute_statement(statement, environment)
85
86     def _execute_statement(self, node: ASTNode, environment:
_Environment) -> None:
87         if isinstance(node, Assign):
88             value = self._evaluate(node.value, environment)
89             environment.assign(node.name, value)
90             return
91
92         if isinstance(node, Print):
93             value = self._evaluate(node.expr, environment)
94             self._output_callback(f"{self._format_value(value)} : {self
._runtime_type(value).value}")
95             return
96
97         if isinstance(node, Return):
98             raise _ReturnSignal(self._evaluate(node.expr, environment))
99
100         if isinstance(node, If):
101             condition = self._evaluate(node.condition, environment)
102             if not isinstance(condition, bool):
103                 raise InterpreterError(self._error_at(node.condition, "
If condition must evaluate to Boolean"))
104             branch = node.then_block if condition else node.else_block
105             if branch is not None:
106                 self._execute_block(branch, _Environment(parent=
environment))
107             return
108
109         if isinstance(node, While):
110             while True:

```

```

111         condition = self._evaluate(node.condition, environment)
112         if not isinstance(condition, bool):
113             raise InterpreterError(self._error_at(node,
114                 condition, "While condition must evaluate to
115                 Boolean"))
114         if not condition:
115             break
116         self._execute_block(node.body, _Environment(parent=
117             environment))
117     return
118
119     if isinstance(node, Block):
120         self._execute_block(node, _Environment(parent=environment))
121         return
122
123     if isinstance(node, FunctionCall):
124         self._evaluate(node, environment)
125         return
126
127     raise InterpreterError(self._error_at(node, f"Unsupported
128         statement node: {type(node).__name__}"))
129
130 def _execute_block(self, block: Block, environment: _Environment)
131     -> None:
132     for statement in block.statements:
133         self._execute_statement(statement, environment)
134
135 def _evaluate(self, node: ASTNode, environment: _Environment) ->
136     object:
137     if isinstance(node, Literal):
138         return node.value
139
140     if isinstance(node, Identifier):
141         return environment.get(node.name)
142
143     if isinstance(node, BinaryOp):
144         left_value = self._evaluate(node.left, environment)
145         right_value = self._evaluate(node.right, environment)
146         return self._evaluate_binary(node, left_value, right_value)
147
148     if isinstance(node, FunctionCall):
149         return self._call_function(node, environment)
150
151     raise InterpreterError(self._error_at(node, f"Unsupported
152         expression node: {type(node).__name__}"))
153
154 def _call_function(self, node: FunctionCall, environment:
155     _Environment) -> object:
156     function_runtime = self._functions.get(node.name)
157     if function_runtime is None:
158         raise InterpreterError(self._error_at(node, f"Undefined
159             function: {node.name}"))

```

```

155     if len(node.args) != len(function_runtime.node.params):
156         raise InterpreterError(
157             self._error_at(
158                 node,
159                 f"Function '{node.name}' expects {len(
160                     function_runtime.node.params)} argument(s), got
161                     {len(node.args)}",
162                 )
163             )
164
165     call_environment = _Environment(parent=environment)
166     argument_values = [self._evaluate(argument, environment) for
167                         argument in node.args]
168     for parameter_name, argument_value in zip(function_runtime.node
169                                               .params, argument_values, strict=False):
170         call_environment.define(parameter_name, argument_value)
171
172     try:
173         self._execute_block(function_runtime.node.body,
174                             call_environment)
175     except _ReturnSignal as signal:
176         return signal.value
177     return None
178
179 def _evaluate_binary(self, node: BinaryOp, left_value: object,
180                     right_value: object) -> object:
181     if node.op == "+":
182         return left_value + right_value
183     if node.op == "-":
184         return left_value - right_value
185     if node.op == "*":
186         return left_value * right_value
187     if node.op == "/":
188         if int(right_value) == 0:
189             raise InterpreterError(self._error_at(node, "Division
190                 by zero"))
191         return int(left_value) // int(right_value)
192     if node.op == "+.":
193         return float(left_value) + float(right_value)
194     if node.op == "-.":
195         return float(left_value) - float(right_value)
196     if node.op == ".*":
197         return float(left_value) * float(right_value)
198     if node.op == "/.":
199         if float(right_value) == 0.0:
200             raise InterpreterError(self._error_at(node, "Division
201                 by zero"))
202         return float(left_value) / float(right_value)
203     if node.op == "==":
204         return left_value == right_value
205     if node.op == "!=":
206         return left_value != right_value
207     raise InterpreterError(self._error_at(node, f"Unsupported

```

```

        operator: {node.op}"))

200
    @staticmethod
201
    def _format_value(value: object) -> str:
202
        if isinstance(value, str):
203
            return f'"{value}"'
204
        if isinstance(value, bool):
205
            return "true" if value else "false"
206
        return str(value)
207
208
    @staticmethod
209
    def _runtime_type(value: object) -> LanguageType:
210
        if isinstance(value, bool):
211
            return LanguageType.BOOLEAN
212
        if isinstance(value, int):
213
            return LanguageType.INTEGER
214
        if isinstance(value, float):
215
            return LanguageType.FLOAT
216
        if isinstance(value, str):
217
            return LanguageType.STRING
218
        return LanguageType.VOID
219
220
    @staticmethod
221
    def _error_at(node: ASTNode, message: str) -> str:
222
        return f"[line {node.line}, col {node.column}] RuntimeError: {
223
            message}"

```

Listing 8.9: components/interpreter.py — tree-walking interpreter

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  from components.ast_printer import ASTPrinter
6  from components.interpreter import Interpreter
7  from components.lexica import Lexer
8  from components.parser import Parser
9  from components.symbol_table import SymbolTable
10 from components.tokens import Token
11 from components.type_checker import TypeChecker
12
13
14 @dataclass
15 class PipelineResult:
16     tokens: list[Token]
17     ast_text: str
18     checked_scope: SymbolTable
19     outputs: list[str]
20
21
22 def run_pipeline(source_code: str) -> PipelineResult:
23     tokens = Lexer(source_code).tokenize()
24     tree = Parser(tokens).parse()

```

```

25     ast_text = ASTPrinter().print(tree)
26     checked_scope = TypeChecker().check(tree)
27
28     outputs: list[str] = []
29     Interpreter(output_callback=outputs.append).execute(tree)
30
31     return PipelineResult(
32         tokens=tokens,
33         ast_text=ast_text,
34         checked_scope=checked_scope,
35         outputs=outputs,
36     )
37
38
39 def format_tokens(tokens: list[Token]) -> str:
40     lines = []
41     for token in tokens:
42         lines.append(
43             f"    {token.token_type.name:<14} {token.lexeme!r:<12} line={
44                 token.line} col={token.column}"
45         )
46     return "\n".join(lines)
47
48 def format_stage_output(result: PipelineResult) -> str:
49     sections = [
50         "=" * 60,
51         "    STAGE 1 -- TOKENS",
52         "=" * 60,
53         format_tokens(result.tokens),
54         "",
55         "=" * 60,
56         "    STAGE 2 -- AST (parser output)",
57         "=" * 60,
58         result.ast_text,
59         "",
60         "=" * 60,
61         "    STAGE 3 -- TYPE CHECK",
62         "=" * 60,
63         result.checked_scope.format_table(),
64         "",
65         "=" * 60,
66         "    STAGE 4 -- EXECUTION",
67         "=" * 60,
68         "\n".join(result.outputs) if result.outputs else "(no output)",
69     ]
70     return "\n".join(sections)

```

Listing 8.10: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```

1 from __future__ import annotations
2
3 import unittest
4
5 from components.interpreter import InterpreterError
6 from components.lexica import Lexer, LexerError
7 from components.parser import Parser, ParseError
8 from components.pipeline import run_pipeline
9 from components.symbol_table import LanguageType, SymbolTable,
    SymbolTableError
10 from components.tokens import TokenType
11 from components.type_checker import TypeCheckError
12
13
14 #
    -----
15 # Lexer
16 #
    -----
17
18 class LexerTests(unittest.TestCase):
19     """Tests for components/lexica.py -- lexical analysis and
        tokenisation."""
20
21     def test_integer_literal_token(self) -> None:
22         tokens = Lexer("42").tokenize()
23         self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
24         self.assertEqual(tokens[0].lexeme, "42")
25
26     def test_float_literal_token(self) -> None:
27         tokens = Lexer("3.14").tokenize()
28         self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
29         self.assertEqual(tokens[0].lexeme, "3.14")
30
31     def test_bool_literals_produce_bool_literal_tokens(self) -> None:
32         tokens = Lexer("true false").tokenize()
33         self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
34         self.assertEqual(tokens[0].lexeme, "true")
35         self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
36         self.assertEqual(tokens[1].lexeme, "false")
37
38     def test_keywords_produce_correct_token_types(self) -> None:
39         tokens = Lexer("if else while def return print").tokenize()
40         expected = [
41             TokenType.IF, TokenType.ELSE, TokenType.WHILE,
42             TokenType.DEF, TokenType.RETURN, TokenType.PRINT,
43         ]
44         actual = [t.token_type for t in tokens if t.token_type !=
                     TokenType.EOF]
45         self.assertEqual(actual, expected)

```

```

46
47 def test_identifier_not_confused_with_keyword(self) -> None:
48     tokens = Lexer("iffy whileloop").tokenize()
49     self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
50     self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)
51
52 def test_two_character_operators(self) -> None:
53     tokens = Lexer("== !=").tokenize()
54     self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
55     self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)
56
57 def test_float_dot_operators_tokenised(self) -> None:
58     tokens = Lexer("+. -. *. /.").tokenize()
59     expected = [TokenType.PLUS_DOT, TokenType.MINUS_DOT, TokenType.
60                 STAR_DOT, TokenType.SLASH_DOT]
61     actual = [t.token_type for t in tokens if t.token_type !=
62               TokenType.EOF]
63     self.assertEqual(actual, expected)
64
65 def test_float_literal_not_confused_with_dot_operator(self) -> None
66 :
67     # "3.14" must produce FLOAT_LITERAL, not INT_LITERAL followed
68     # by something
69     tokens = Lexer("3.14").tokenize()
70     self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
71     self.assertEqual(tokens[0].lexeme, "3.14")
72
73 def test_source_position_tracked_across_lines(self) -> None:
74     tokens = Lexer("x\ny").tokenize()
75     self.assertEqual(tokens[0].line, 1)
76     self.assertEqual(tokens[1].line, 2)
77
78 def test_unexpected_character_raises_lexer_error(self) -> None:
79     with self.assertRaises(LexerError):
80         Lexer("@").tokenize()
81
82 def test_lone_exclamation_raises_lexer_error(self) -> None:
83     with self.assertRaises(LexerError):
84         Lexer("!5").tokenize()
85
86 def test_unterminated_string_raises_lexer_error(self) -> None:
87     with self.assertRaises(LexerError):
88         Lexer('"hello').tokenize()
89
90 #
91
92 # Symbol table
93 #

```

```

91 class SymbolTableTests(unittest.TestCase):
92     """Tests for components/symbol_table.py -- scoped symbol table and
        semantic types."""
93
94     def test_define_variable_and_lookup(self) -> None:
95         table = SymbolTable()
96         table.define_variable("x", LanguageType.INTEGER, initialized=
            True)
97         symbol = table.lookup("x")
98         self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
99
100    def test_duplicate_variable_definition_raises_error(self) -> None:
101        table = SymbolTable()
102        table.define_variable("x", LanguageType.INTEGER)
103        with self.assertRaises(SymbolTableError):
104            table.define_variable("x", LanguageType.FLOAT)
105
106    def test_duplicate_function_definition_raises_error(self) -> None:
107        table = SymbolTable()
108        table.define_function("f", LanguageType.INTEGER, [])
109        with self.assertRaises(SymbolTableError):
110            table.define_function("f", LanguageType.FLOAT, [])
111
112    def test_lookup_in_parent_scope(self) -> None:
113        parent = SymbolTable()
114        parent.define_variable("x", LanguageType.INTEGER, initialized=
            True)
115        child = parent.child_scope()
116        symbol = child.lookup("x")
117        self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)
118
119    def test_undefined_symbol_raises_error(self) -> None:
120        table = SymbolTable()
121        with self.assertRaises(SymbolTableError):
122            table.lookup("z")
123
124    def test_format_table_contains_variable_row(self) -> None:
125        table = SymbolTable()
126        table.define_variable("counter", LanguageType.INTEGER,
            initialized=True)
127        output = table.format_table()
128        self.assertIn("counter", output)
129        self.assertIn("variable", output)
130        self.assertIn("Integer", output)
131
132    def test_format_table_contains_function_row(self) -> None:
133        table = SymbolTable()
134        table.define_function("add", LanguageType.INTEGER, [("a",
            LanguageType.INTEGER)])
135        output = table.format_table()
136        self.assertIn("add", output)
137        self.assertIn("function", output)
138

```

```

139
140 #
141
142 -----
143
144 # Parser
145 #
146 -----
147
148 class ParserTests(unittest.TestCase):
149     """Tests for components/parser.py -- recursive-descent parsing."""
150
151     def _parse(self, source: str):
152         return Parser(Lexer(source).tokenize()).parse()
153
154     def test_missing_semicolon_raises_parse_error(self) -> None:
155         with self.assertRaises(ParseError):
156             self._parse("x = 5")
157
158     def test_unclosed_brace_raises_parse_error(self) -> None:
159         with self.assertRaises(ParseError):
160             self._parse("if (true) { x = 1;}")
161
162     def test_operator_precedence_mul_before_add(self) -> None:
163         # 2 + 3 * 4 must be 14, not 20
164         result = run_pipeline("x = 2 + 3 * 4; print(x);")
165         self.assertEqual(result.outputs, ["14 : Integer"])
166
167     def test_operator_left_associativity(self) -> None:
168         # 10 - 3 - 2 must be 5 (left-associ), not 9
169         result = run_pipeline("x = 10 - 3 - 2; print(x);")
170         self.assertEqual(result.outputs, ["5 : Integer"])
171
172     def test_parentheses_override_precedence(self) -> None:
173         # (2 + 3) * 4 must be 20
174         result = run_pipeline("x = (2 + 3) * 4; print(x);")
175         self.assertEqual(result.outputs, ["20 : Integer"])
176
177     def test_float_operator_precedence_mul_before_add(self) -> None:
178         # 1.0 +. 2.0 *. 3.0 must be 7.0 (*. binds tighter than +.)
179         result = run_pipeline("x = 1.0 +. 2.0 *. 3.0; print(x);")
180         self.assertEqual(result.outputs, ["7.0 : Float"])
181
182 #
183 -----
184
185 # Type checker
186 #
187 -----
188
189 class TypeCheckerTests(unittest.TestCase):

```

```

184 """Tests for components/type_checker.py -- static type inference and
185    checking."""
186
187 def test_integer_arithmetic_stays_integer(self) -> None:
188     result = run_pipeline("x = 3 + 4; print(x);")
189     self.assertEqual(result.outputs, ["7 : Integer"])
190
191 def test_division_always_produces_integer(self) -> None:
192     # integer / integer -> Integer (no implicit float conversion)
193     result = run_pipeline("x = 10 / 2; print(x);")
194     self.assertEqual(result.outputs, ["5 : Integer"])
195
196 def test_float_dot_operators_produce_float(self) -> None:
197     result = run_pipeline("x = 1.0 +. 2.5; print(x);")
198     self.assertEqual(result.outputs, ["3.5 : Float"])
199
200 def test_float_subtraction_produces_float(self) -> None:
201     result = run_pipeline("x = 5.0 -. 1.5; print(x);")
202     self.assertEqual(result.outputs, ["3.5 : Float"])
203
204 def test_float_multiplication_produces_float(self) -> None:
205     result = run_pipeline("x = 2.0 *. 3.0; print(x);")
206     self.assertEqual(result.outputs, ["6.0 : Float"])
207
208 def test_float_division_produces_float(self) -> None:
209     result = run_pipeline("x = 9.0 /. 4.0; print(x);")
210     self.assertEqual(result.outputs, ["2.25 : Float"])
211
212 def test_integer_with_float_op_raises_type_error(self) -> None:
213     # +. requires Float operands; integers must not be accepted
214     with self.assertRaises(TypeError):
215         run_pipeline("x = 2 +. 3; print(x);")
216
217 def test_mixed_int_float_addition_raises_type_error(self) -> None:
218     # Integer + Float is no longer valid; must use +.
219     with self.assertRaises(TypeError):
220         run_pipeline("x = 1 + 2.5; print(x);")
221
222 def test_string_variable_type_inferred(self) -> None:
223     result = run_pipeline('x = "hello"; print(x);')
224     self.assertEqual(result.outputs, ['"hello" : String'])
225
226 def test_boolean_literal_type_inferred(self) -> None:
227     result = run_pipeline("x = true; print(x);")
228     self.assertEqual(result.outputs, ["true : Boolean"])
229
230 def test_boolean_equality_raises_type_error(self) -> None:
231     # Boolean == Boolean is not allowed; == only accepts arithmetic
232     operands
233     with self.assertRaises(TypeError):
234         run_pipeline('x = true; if (x == false) { print("then"); }
235             else { print("else"); }')

```

```

234 def test_mixed_int_float_equality_raises_type_error(self) -> None:
235     # == and != must not accept mixed Integer/Float (no overloading
236     )
237     with self.assertRaises(TypeError):
238         run_pipeline("x = 5; y = 3.14; if (x == y) { print(x); }")
239
240 def test_mixed_int_float_inequality_raises_type_error(self) -> None:
241     with self.assertRaises(TypeError):
242         run_pipeline("x = 5; y = 3.14; if (x != y) { print(x); }")
243
244 def test_arithmetic_on_string_raises_type_error(self) -> None:
245     with self.assertRaises(TypeError):
246         run_pipeline('x = "a" + 1;')
247
248 def test_if_non_boolean_condition_raises_type_error(self) -> None:
249     with self.assertRaises(TypeError):
250         run_pipeline("if (1) { x = 2; }")
251
252 def test_while_non_boolean_condition_raises_type_error(self) ->
253 None:
254     with self.assertRaises(TypeError):
255         run_pipeline("x = 0; while (x) { x = x + 1; }")
256
257 def test_assignment_type_mismatch_raises_type_error(self) -> None:
258     with self.assertRaises(TypeError):
259         run_pipeline('x = 5; x = "hello";')
260
261 def test_undefined_variable_raises_type_error(self) -> None:
262     with self.assertRaises(TypeError):
263         run_pipeline("print(z);")
264
265 def test_function_wrong_arg_count_raises_type_error(self) -> None:
266     with self.assertRaises(TypeError):
267         run_pipeline("def f(a) { return a; } f(1, 2);")
268
269 def test_function_arg_type_mismatch_raises_type_error(self) -> None:
270     :
271     # first call infers a: Integer; second call with String must
272     fail
273     with self.assertRaises(TypeError):
274         run_pipeline('def f(a) { return a; } f(1); f("hello");')
275
276 def test_uncalled_function_body_type_error_is_caught(self) -> None:
277     # type error inside body must be detected even if function is
278     never called
279     with self.assertRaises(TypeError):
280         run_pipeline('def bad() { y = "hello" + 1; }')
281
282 def test_valid_uncalled_function_does_not_raise(self) -> None:
283     # a well-typed but uncalled function should not raise
284     result = run_pipeline("def add(a, b) { return a + b; }")
285     self.assertEqual(result.outputs, [])

```

```

281
282
283 #
284 # Integration (full pipeline)
285 #
286
287 class IntegrationTests(unittest.TestCase):
288     """End-to-end tests through all four pipeline stages."""
289
290     # --- Unary minus ---
291
292     def test_unary_minus_integer(self) -> None:
293         result = run_pipeline("x = -5; print(x);")
294         self.assertEqual(result.outputs, ["-5 : Integer"])
295
296     def test_unary_minus_float(self) -> None:
297         result = run_pipeline("y = -. 3.14; print(y);")
298         self.assertEqual(result.outputs, ["-3.14 : Float"])
299
300     def test_unary_minus_dot_variable(self) -> None:
301         result = run_pipeline("x = 1.5; y = -. x; print(y);")
302         self.assertEqual(result.outputs, ["-1.5 : Float"])
303
304     def test_unary_minus_variable(self) -> None:
305         result = run_pipeline("x = 10; y = -x; print(y);")
306         self.assertEqual(result.outputs, ["-10 : Integer"])
307
308     def test_unary_minus_inside_expression(self) -> None:
309         result = run_pipeline("x = 3 + -2; print(x);")
310         self.assertEqual(result.outputs, ["1 : Integer"])
311
312     # --- Control flow ---
313
314     def test_if_then_branch_executes(self) -> None:
315         result = run_pipeline(
316             'x = 5; if (x == 5) { print("yes"); } else { print("no"); }
317             ,
318         )
319         self.assertEqual(result.outputs, ["yes" : String'])
320
321     def test_if_else_branch_executes_when_condition_false(self) -> None:
322         result = run_pipeline(
323             'x = 0; if (x == 5) { print("yes"); } else { print("no"); }
324             ,
325         )
326         self.assertEqual(result.outputs, ["no" : String'])
327
328     def test_while_loop_executes_repeatedly(self) -> None:

```

```

327     result = run_pipeline("x = 3; while (x != 0) { print(x); x = x
328         - 1; }")
329     self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer",
330         "1 : Integer"])
331
332     # --- Division by zero ---
333
334     def test_integer_division_by_zero_raises_error(self) -> None:
335         with self.assertRaises(InterpreterError):
336             run_pipeline("x = 10 / 0; print(x);")
337
338     def test_float_division_by_zero_raises_error(self) -> None:
339         with self.assertRaises(InterpreterError):
340             run_pipeline("x = 10.0 /. 0.0; print(x);")
341
342     # --- Functions ---
343
344     def test_function_integer_return(self) -> None:
345         result = run_pipeline("def square(n) { return n * n; } print(
346             square(7));")
347         self.assertEqual(result.outputs, ["49 : Integer"])
348
349     def test_function_type_inference(self) -> None:
350         result = run_pipeline(
351             "def add(a, b) { return a +. b; } result = add(2.0, 3.5);
352             print(result);"
353         )
354         self.assertEqual(result.outputs, ["5.5 : Float"])
355         self.assertIn("result variable Float", result.
356             checked_scope.format_table())
357
358     def test_function_value_parameter_passing(self) -> None:
359         # Modifying a parameter inside a function must not affect the
360         caller's variable
361         result = run_pipeline(
362             "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x);
363             print(x); print(y);"
364         )
365         self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer
366             "])
367
368     def test_nested_function_calls(self) -> None:
369         result = run_pipeline(
370             "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
371         )
372         self.assertEqual(result.outputs, ["6 : Integer"])
373
374     # --- print() with all four types ---
375
376     def test_print_integer(self) -> None:
377         result = run_pipeline("print(42);")
378         self.assertEqual(result.outputs, ["42 : Integer"])

```

```

372 def test_print_float(self) -> None:
373     result = run_pipeline("print(3.14);")
374     self.assertEqual(result.outputs, ["3.14 : Float"])
375
376 def test_print_boolean(self) -> None:
377     result = run_pipeline("print(true);")
378     self.assertEqual(result.outputs, ["true : Boolean"])
379
380 def test_print_string(self) -> None:
381     result = run_pipeline('print("plc");')
382     self.assertEqual(result.outputs, ['"plc" : String'])
383
384 # --- Boolean expressions as values ---
385
386 def test_comparison_result_assigned_to_variable(self) -> None:
387     result = run_pipeline("x = 5; y = 5; flag = (x == y); print(
388         flag);")
389     self.assertEqual(result.outputs, ["true : Boolean"])
390
391 def test_comparison_result_printed_directly(self) -> None:
392     result = run_pipeline("x = 10; print(x != 0);")
393     self.assertEqual(result.outputs, ["true : Boolean"])
394
395 def test_comparison_result_false(self) -> None:
396     result = run_pipeline("a = 1; b = 2; eq = (a == b); print(eq);"
397         )
398     self.assertEqual(result.outputs, ["false : Boolean"])
399
400 # --- Reassignment ---
401
402 def test_variable_can_be_reassigned_same_type(self) -> None:
403     result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
404     self.assertEqual(result.outputs, ["3 : Integer"])
405
406 def test_scope_isolation_if_block(self) -> None:
407     # Variable defined inside an if-block must not be visible
408     # outside it
409     with self.assertRaises(TypeError):
410         run_pipeline(
411             "x = 1; if (x == 1) { inner = 99; } print(inner);"
412         )
413
414 def test_string_equality_raises_type_error(self) -> None:
415     # == is only valid between arithmetic (Integer/Float) operands
416     with self.assertRaises(TypeError):
417         run_pipeline('a = "hello"; b = "world"; if (a == b) { print
418             (a); }')
419
420 def test_integer_floor_division(self) -> None:
421     # 7 / 2 must be 3 (floor), not 3.5
422     result = run_pipeline("x = 7 / 2; print(x);")
423     self.assertEqual(result.outputs, ["3 : Integer"])

```

```
421     def test_void_function_assignment_raises_type_error(self) -> None:
422         # Assigning the result of a void function (no return) must be a
           type error
423     with self.assertRaises(TypeError):
424         run_pipeline("def greet() { print(42); } x = greet();")
425
426
427 if __name__ == "__main__":
428     unittest.main()
```

Listing 8.11: tests/test_pipeline.py — automated regression suite (73 tests)